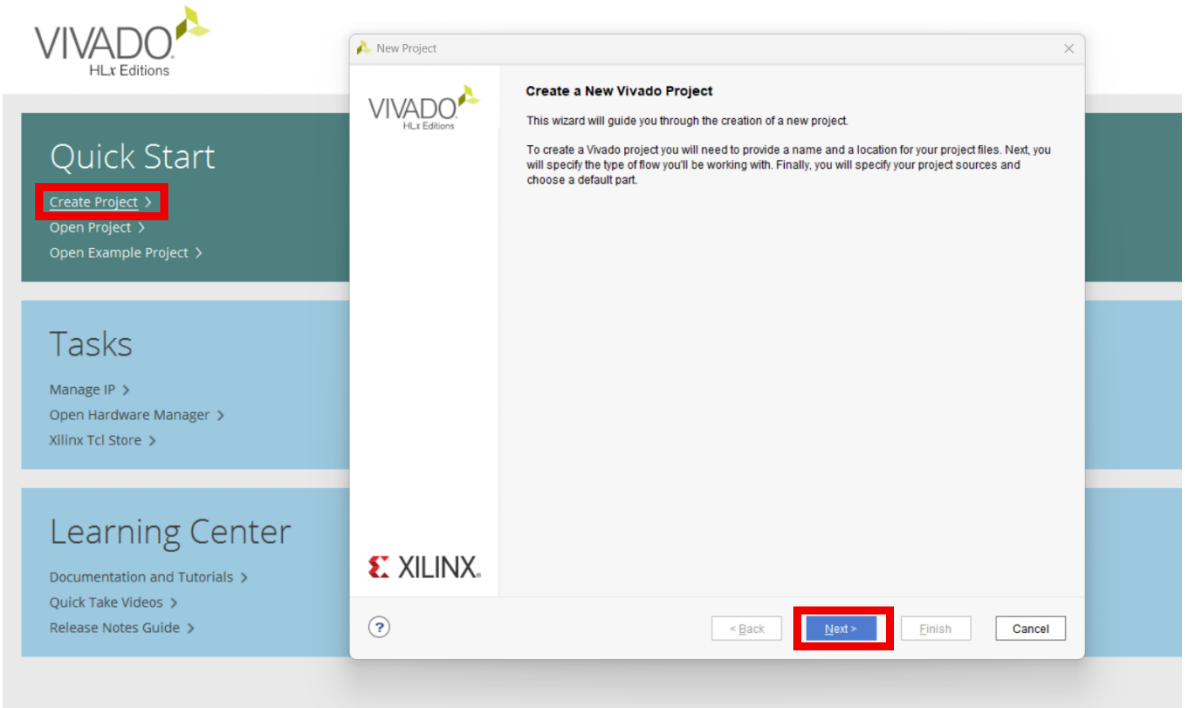
Vivado FPGA Flow

Steps to use Vivado:

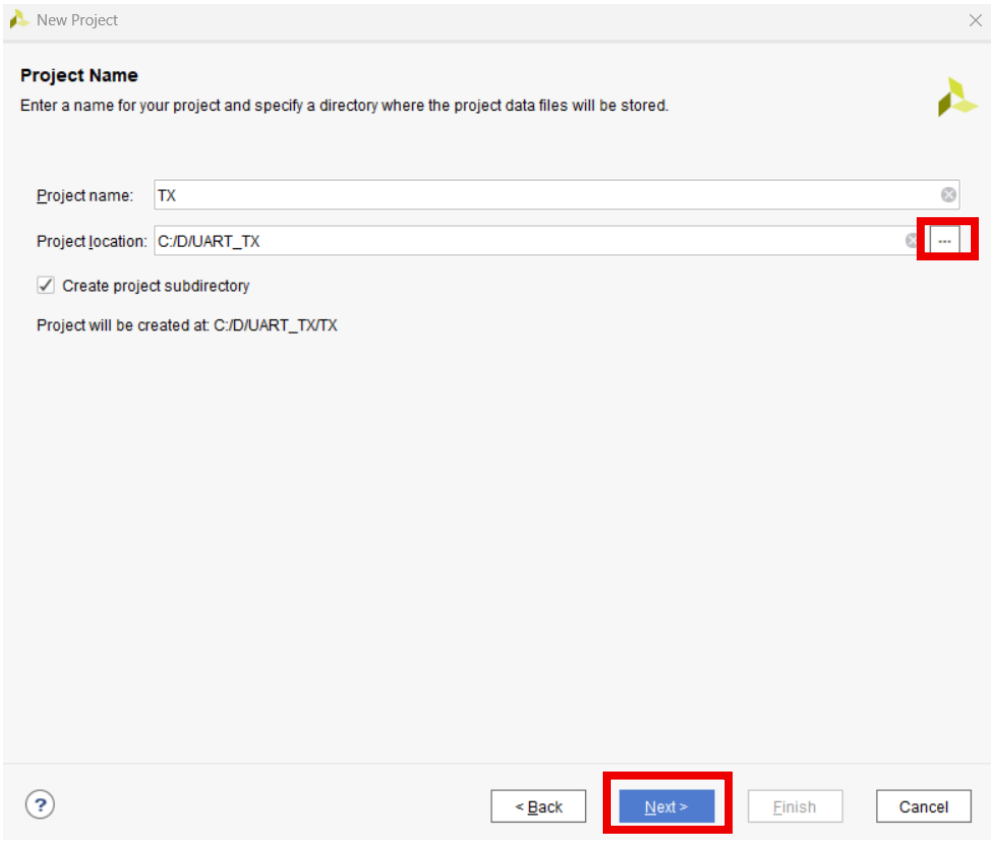
- Launch Vivado and create a new project
- Specify the project's name and its location.
- Select **RTL project** as the project type
- Add your **RTL source files**
- Add your **constraints (XDC) file**
- Select the target FPGA board
- Review the selected project settings and confirm the project creation.
- Run **testbench simulation** to verify the design functionality *(optional)*.
- Run **Elaboration** to check the RTL design and hierarchy.
- Run **Synthesis** to convert the RTL into a synthesized netlist.
- Run **Implementation** to perform placement and routing.
- Generate the **bitstream** and use it to program the FPGA development board.

Steps: -

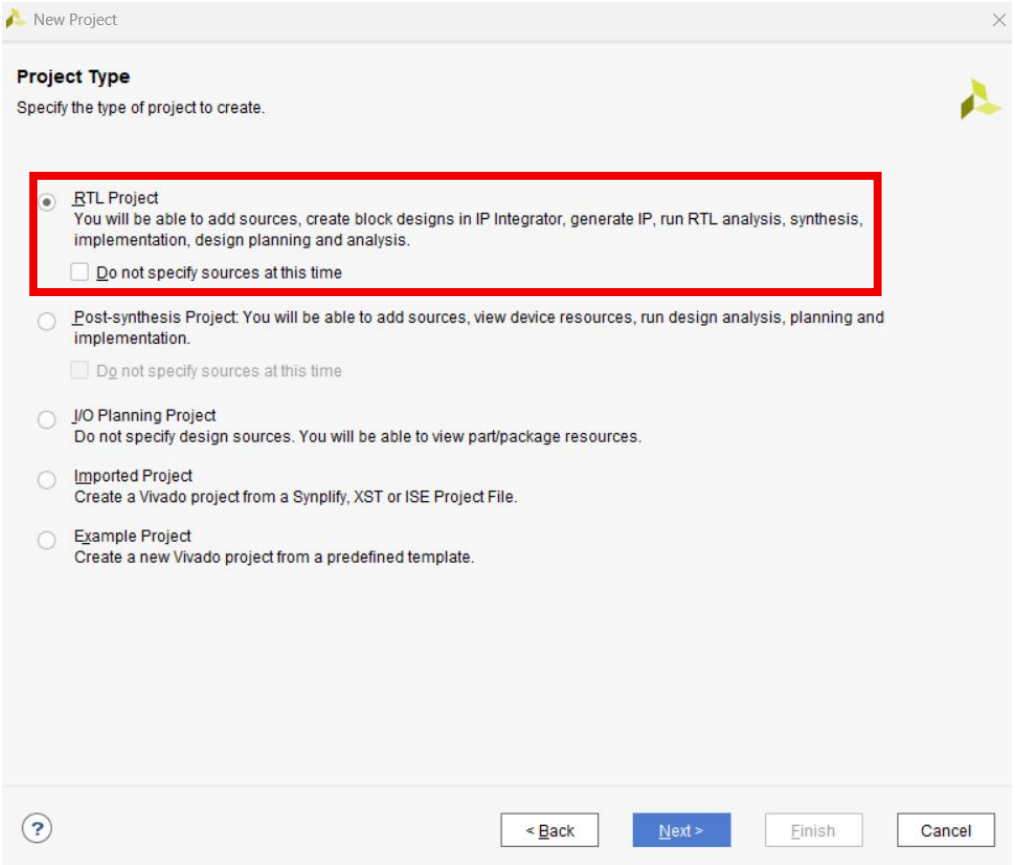
1. Launch Vivado and create a new project



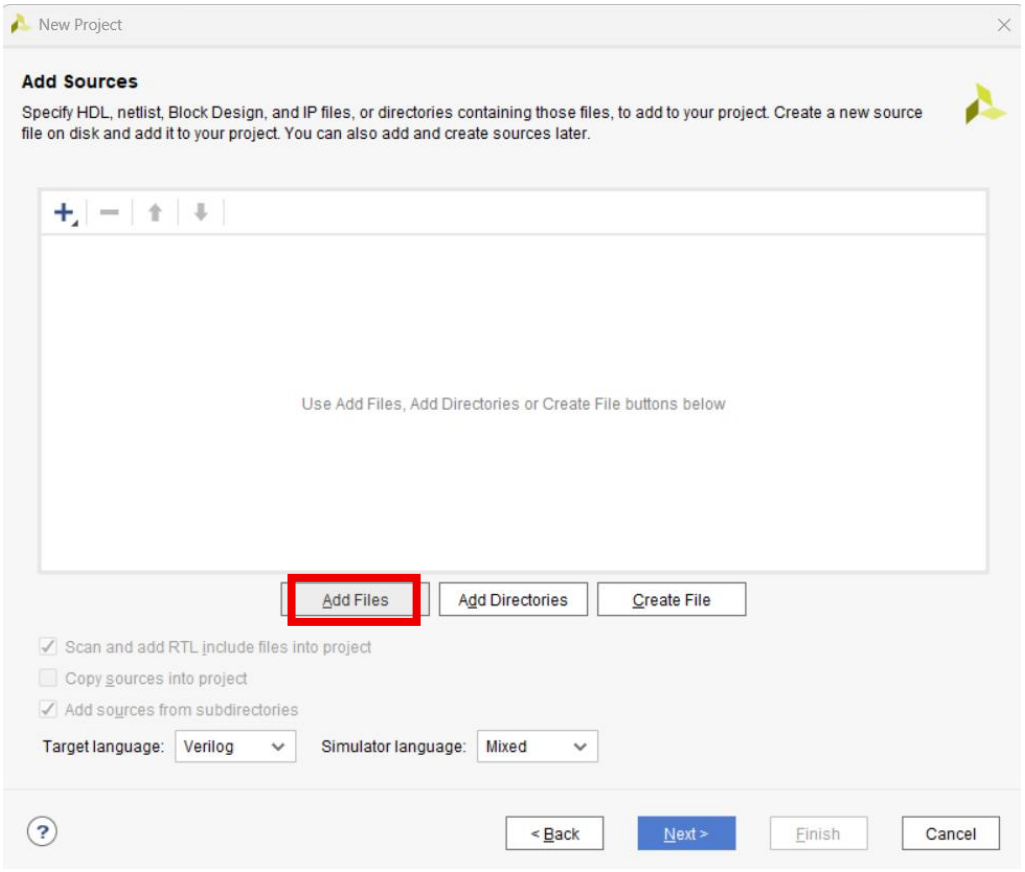
2. Specify the project's name and its location (click on the 3 points to specify the location)



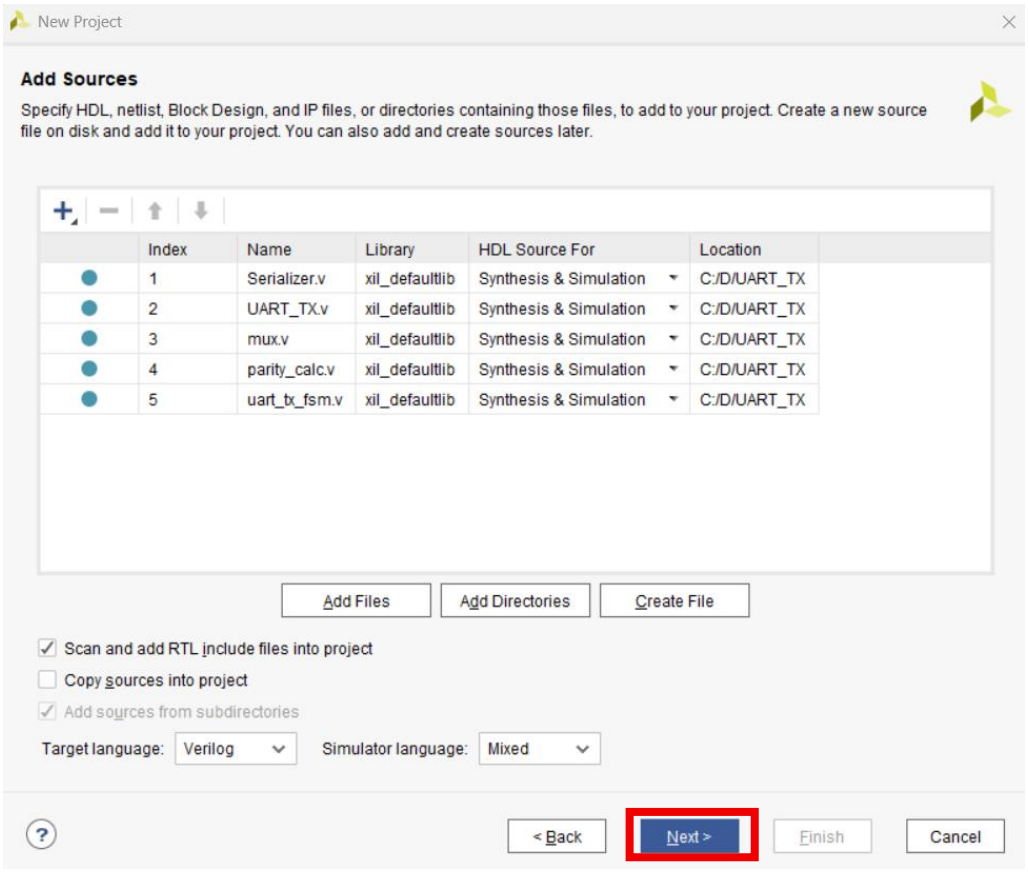
3. Select **RTL project** as the project type



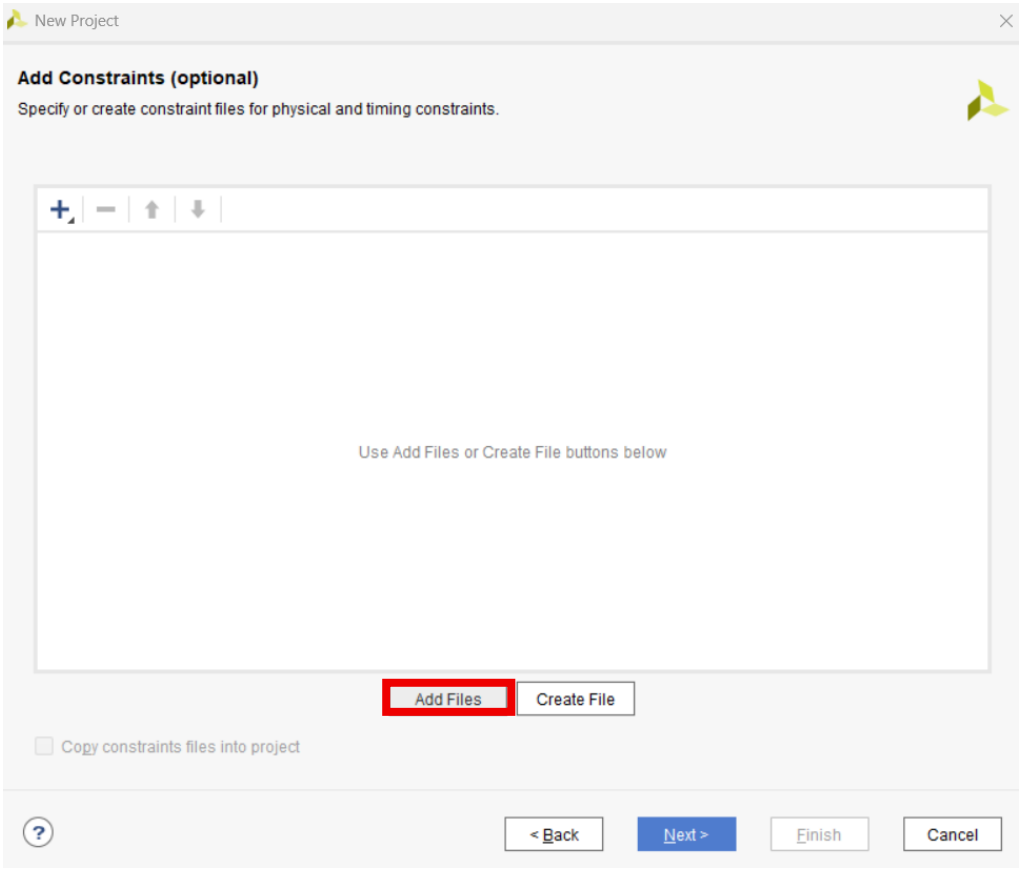
4. Add your **RTL source files** (click on Add Files)



- After adding your RTL files, it will be shown as below:



5. Add your **constraints (XDC) file** (click on Add Files)



- Your constraints file will have the input/output pin mapping and clock constraints. So, it will be like the following Pic:

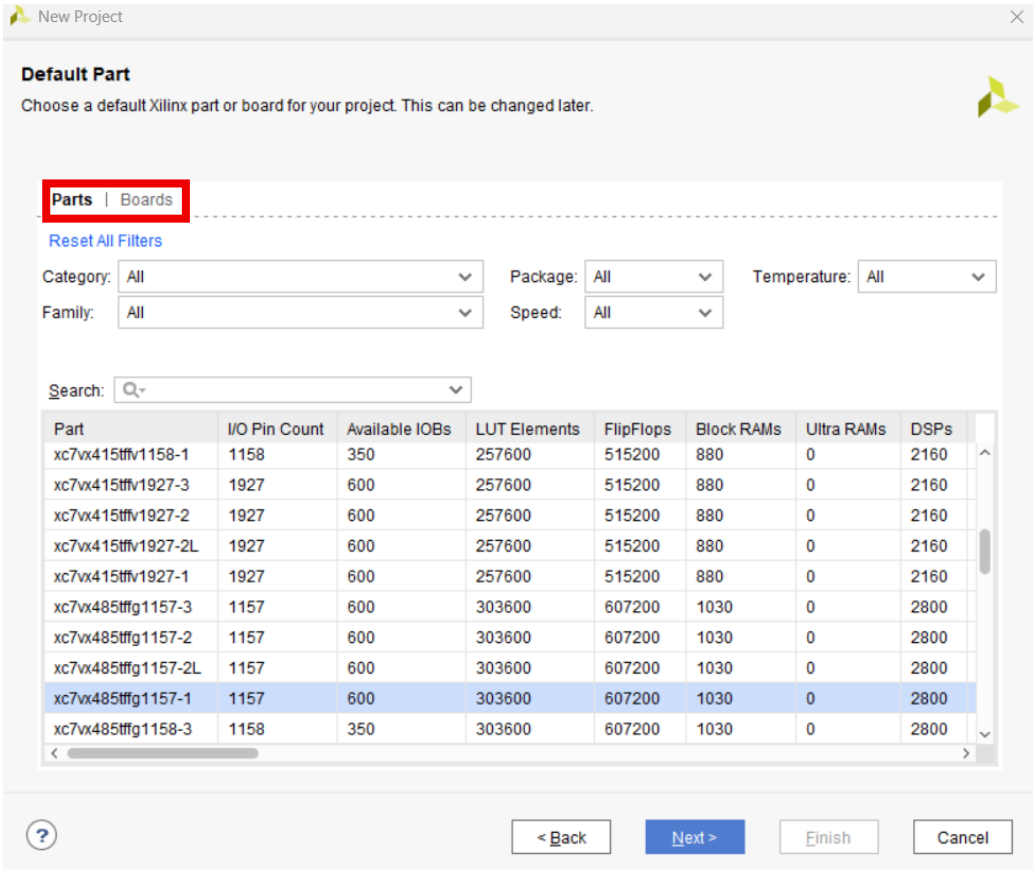
```
## Clock signal set for 115.2 kHz input
set_property -dict { PACKAGE_PIN K17   IOSTANDARD LVCMOS33 } [get_ports { CLK }]; # sysclk
create_clock -add -name sys_clk_pin -period 8680.56 -waveform {0 4340.28} [get_ports { CLK }];

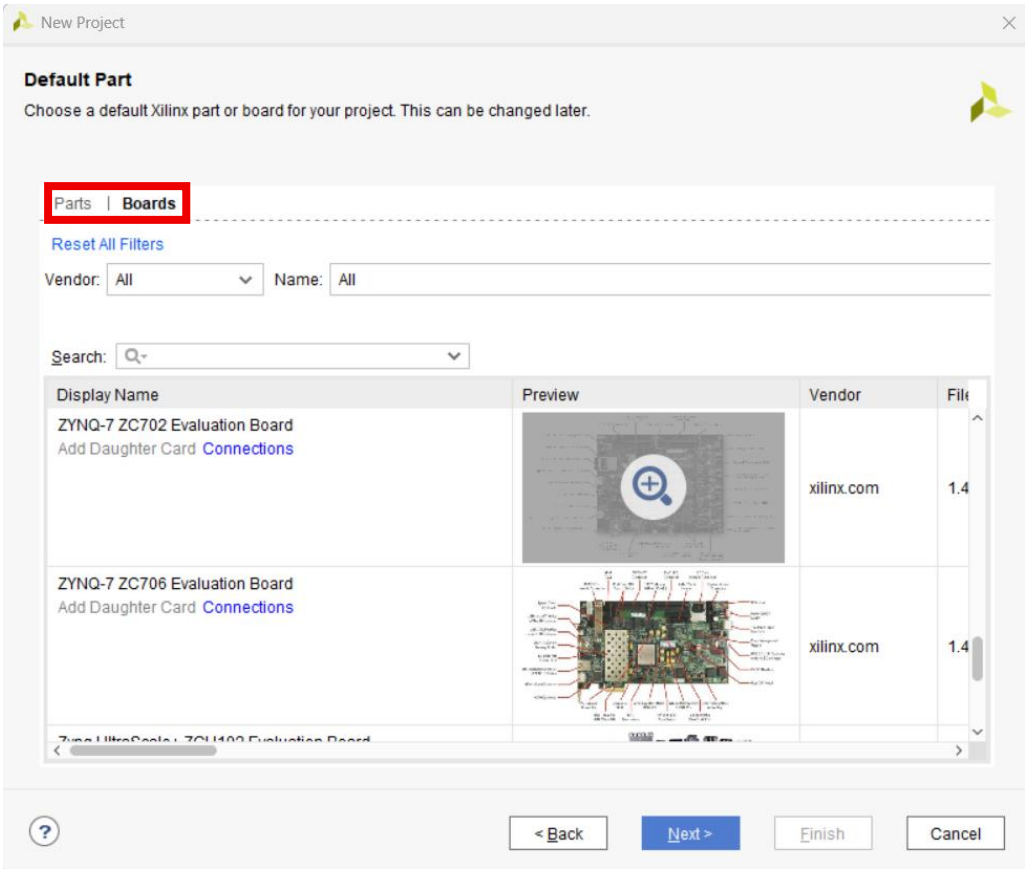
## Control Inputs
set_property -dict { PACKAGE_PIN K18   IOSTANDARD LVCMOS33 } [get_ports { RST }];           # Push Button 0
set_property -dict { PACKAGE_PIN P16   IOSTANDARD LVCMOS33 } [get_ports { Data_Valid }];    # Push Button 1
set_property -dict { PACKAGE_PIN G15   IOSTANDARD LVCMOS33 } [get_ports { parity_enable }];# Switch 0
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { parity_type }];  # Switch 1

## 8-bit Parallel Data Input (P_DATA) mapped to Pmod Header JA
set_property -dict { PACKAGE_PIN N15   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[0] }]; # JA1_P
set_property -dict { PACKAGE_PIN L14   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[1] }]; # JA2_P
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[2] }]; # JA3_P
set_property -dict { PACKAGE_PIN K14   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[3] }]; # JA4_P
set_property -dict { PACKAGE_PIN N16   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[4] }]; # JA1_N
set_property -dict { PACKAGE_PIN L15   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[5] }]; # JA2_N
set_property -dict { PACKAGE_PIN J16   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[6] }]; # JA3_N
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { P_DATA[7] }]; # JA4_N

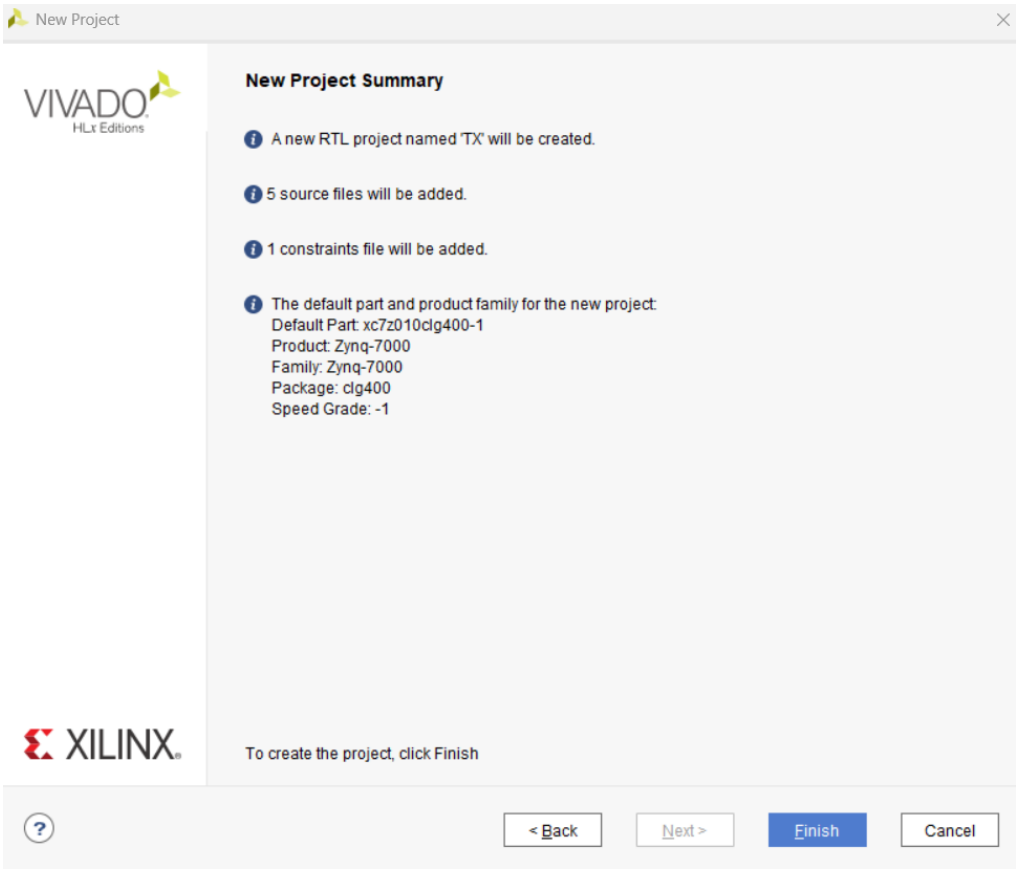
## Outputs
set_property -dict { PACKAGE_PIN M14   IOSTANDARD LVCMOS33 } [get_ports { TX_OUT }]; # LED 0
set_property -dict { PACKAGE_PIN M15   IOSTANDARD LVCMOS33 } [get_ports { busy }];   # LED 1
```

6. Select the target FPGA board (part number or board)

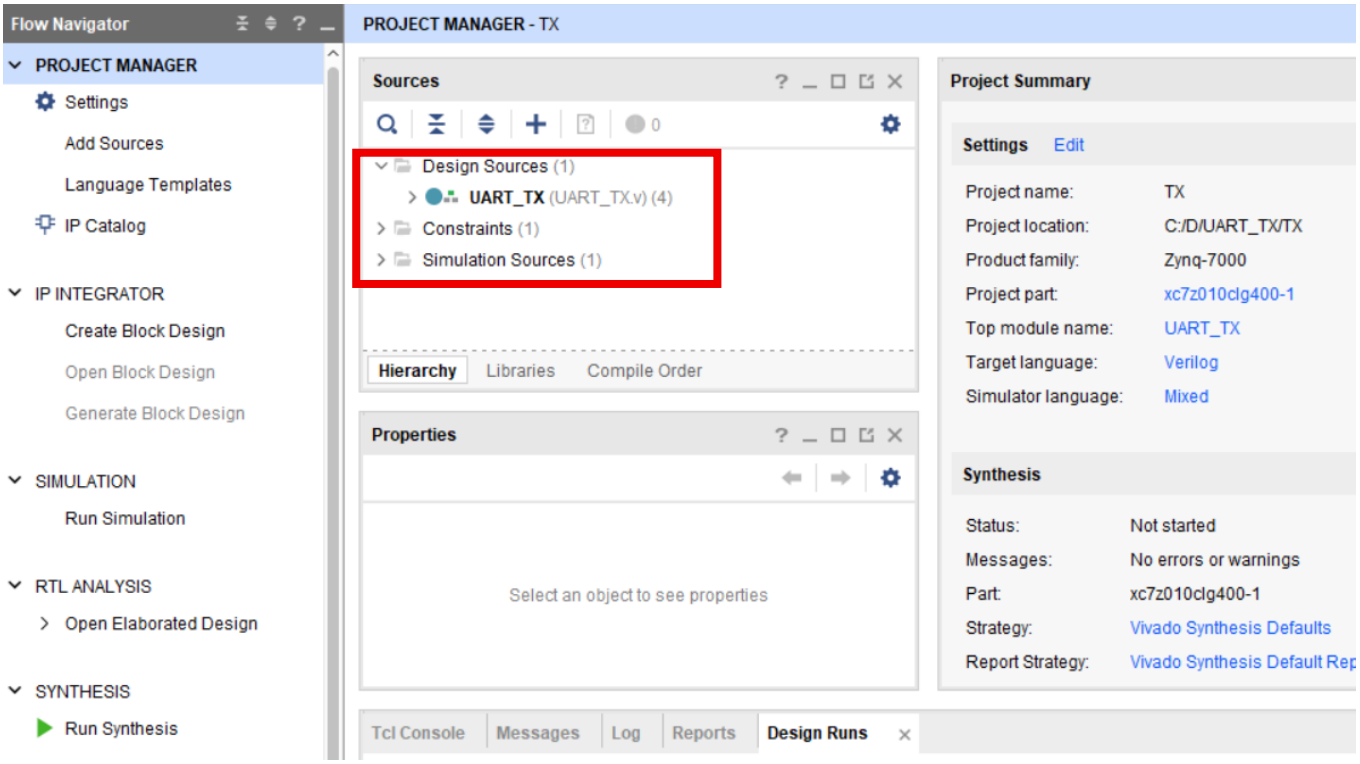




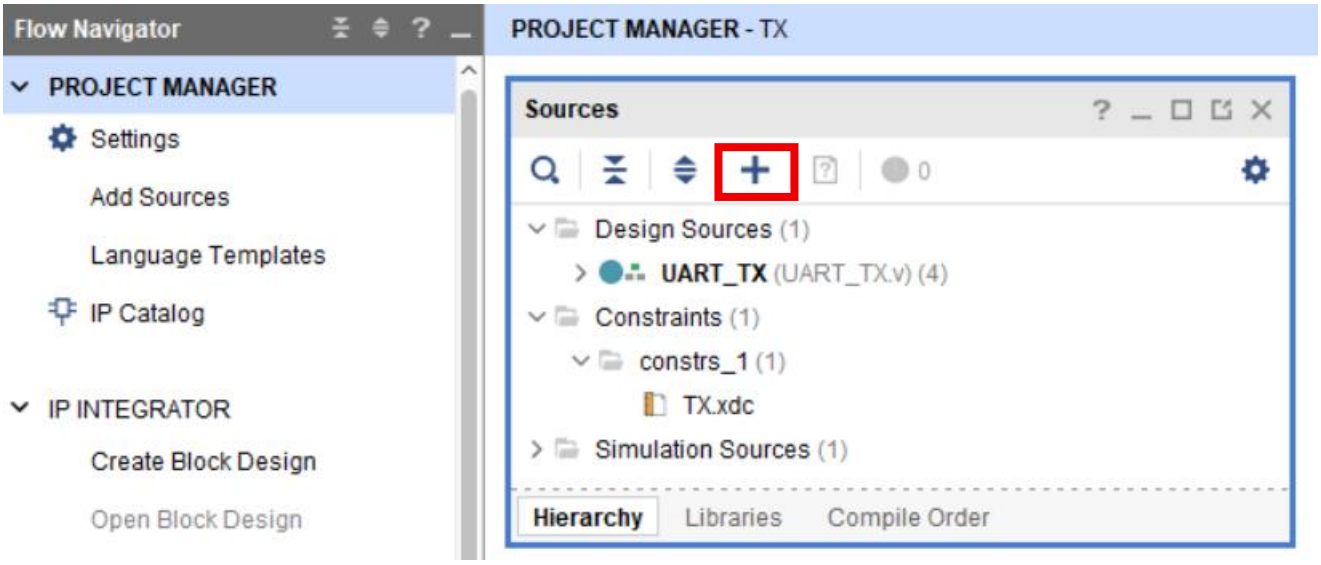
7. Review of the selected project settings (click finish after your confirmation)



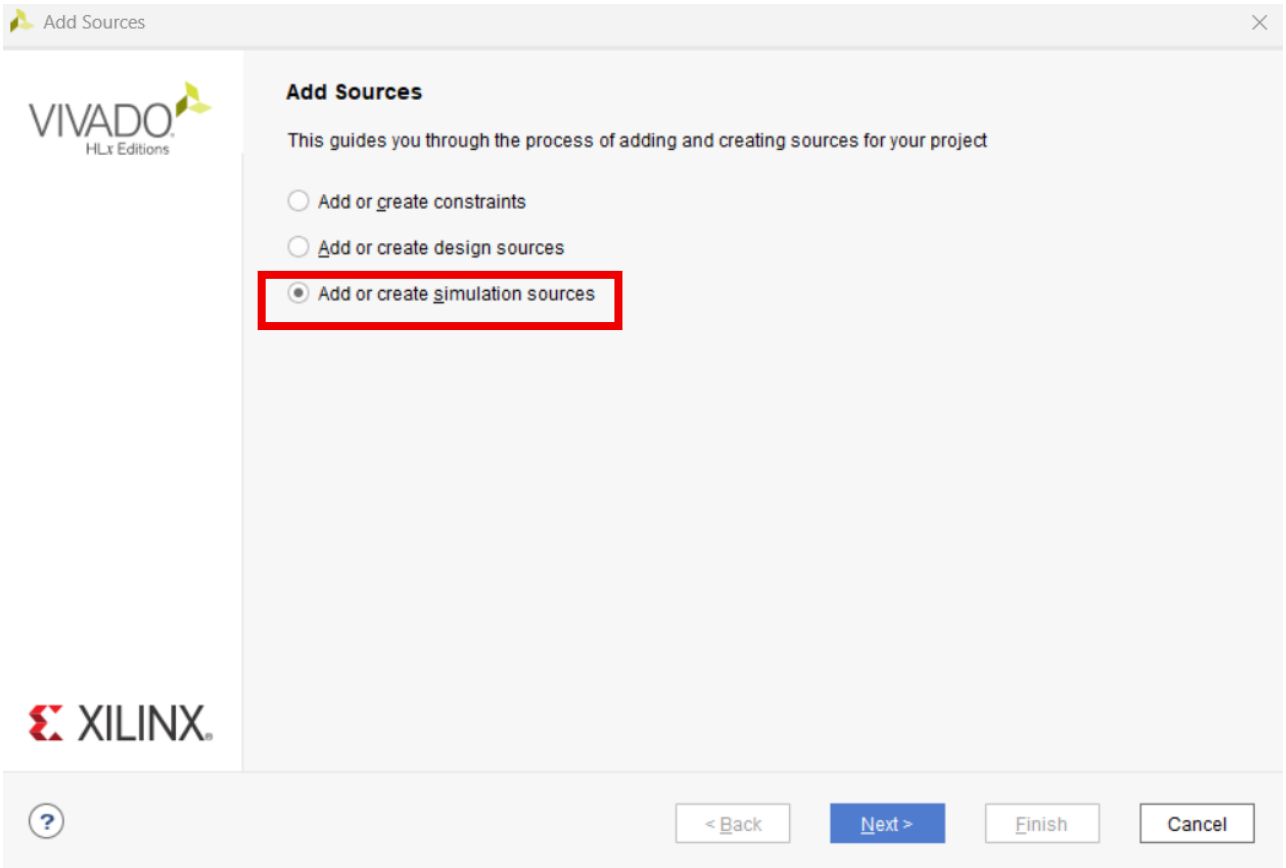
- The next window will be shown as the main interface for Vivado



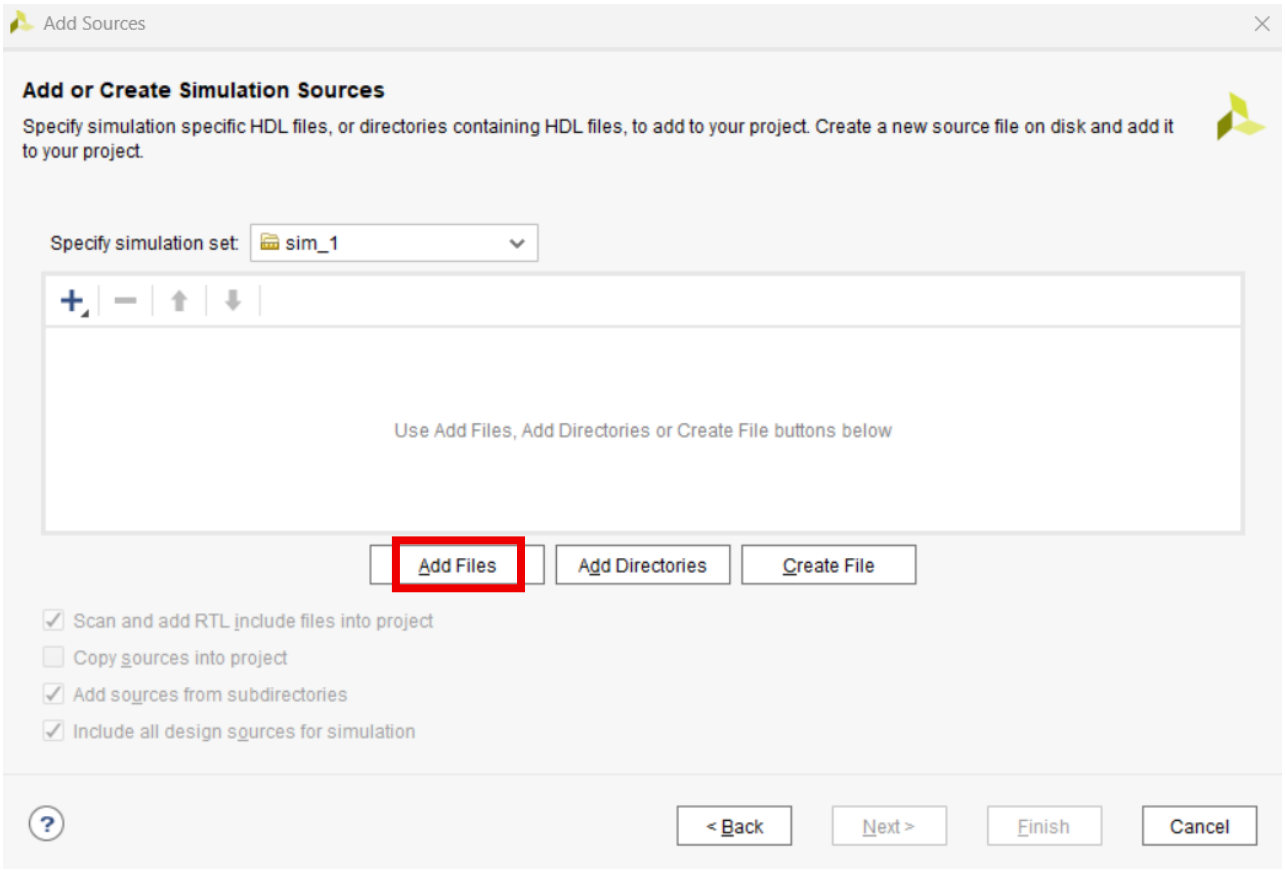
8. Add the testbench file to run **simulation** (click on “**+**” sign)



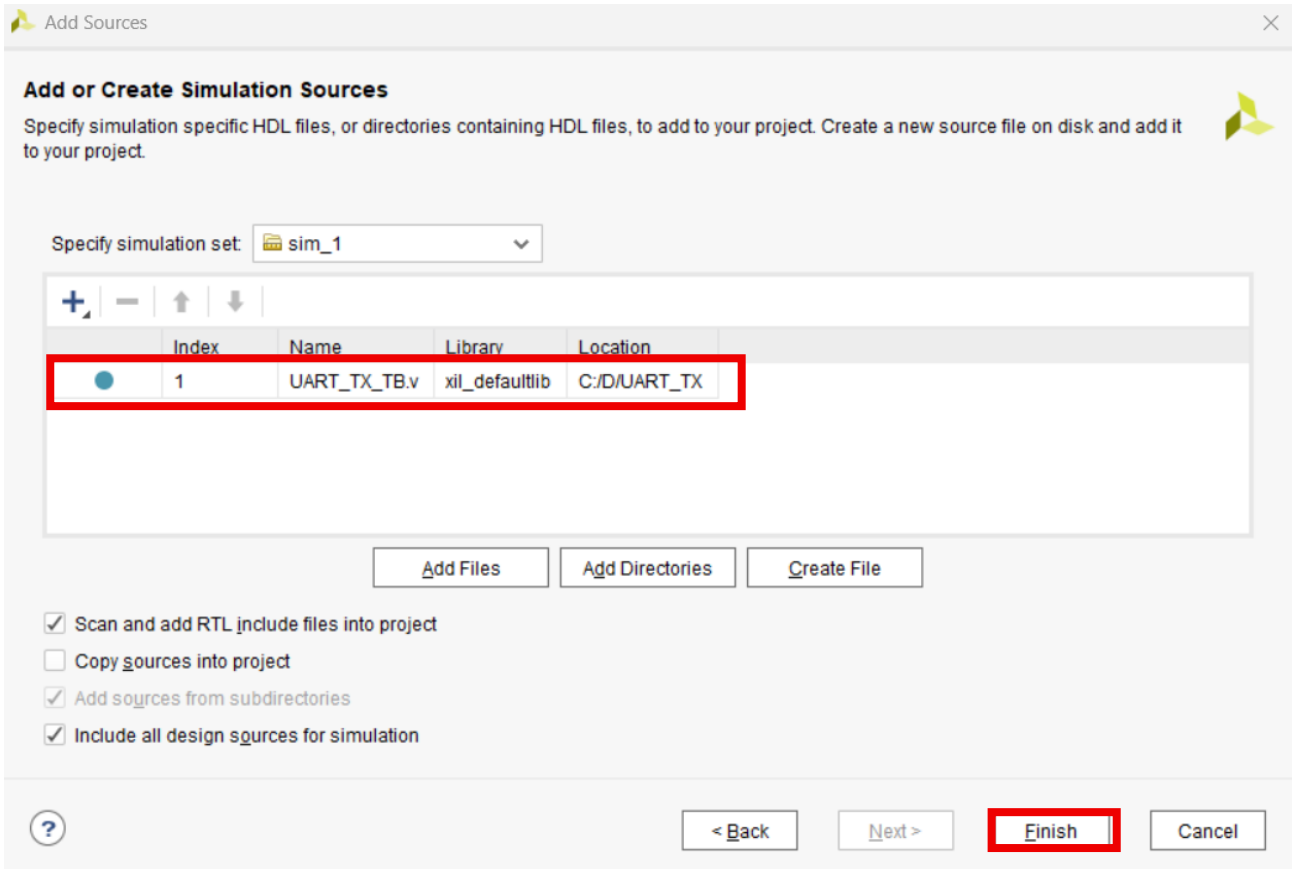
- The Next window will open (select simulation sources and click Next)



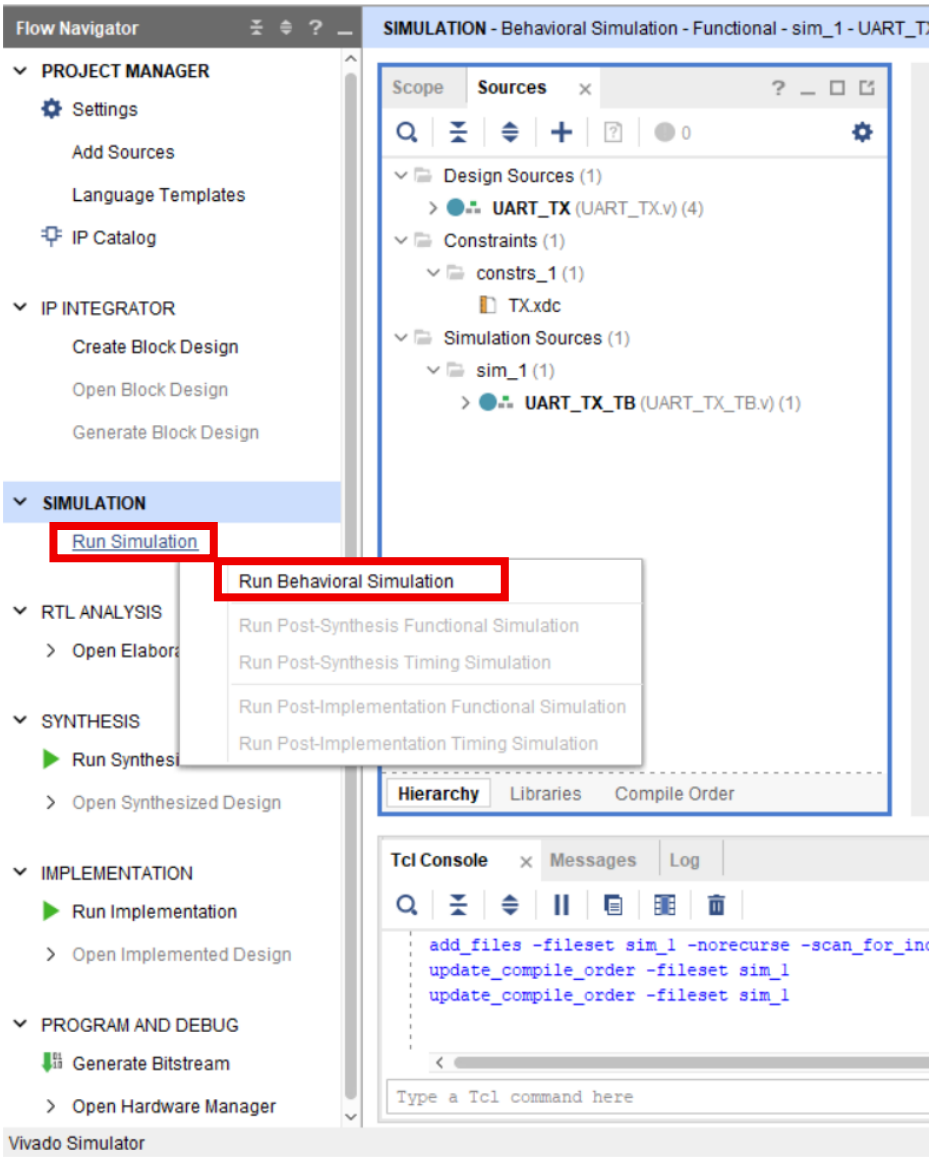
- The Next window will open (Add your test bench file and click Next)



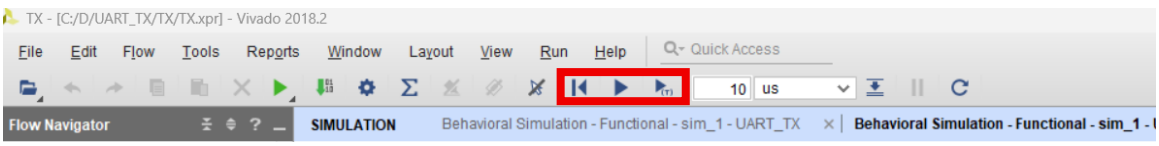
- After you select it, it will be shown as follows (click finish after confirmation):



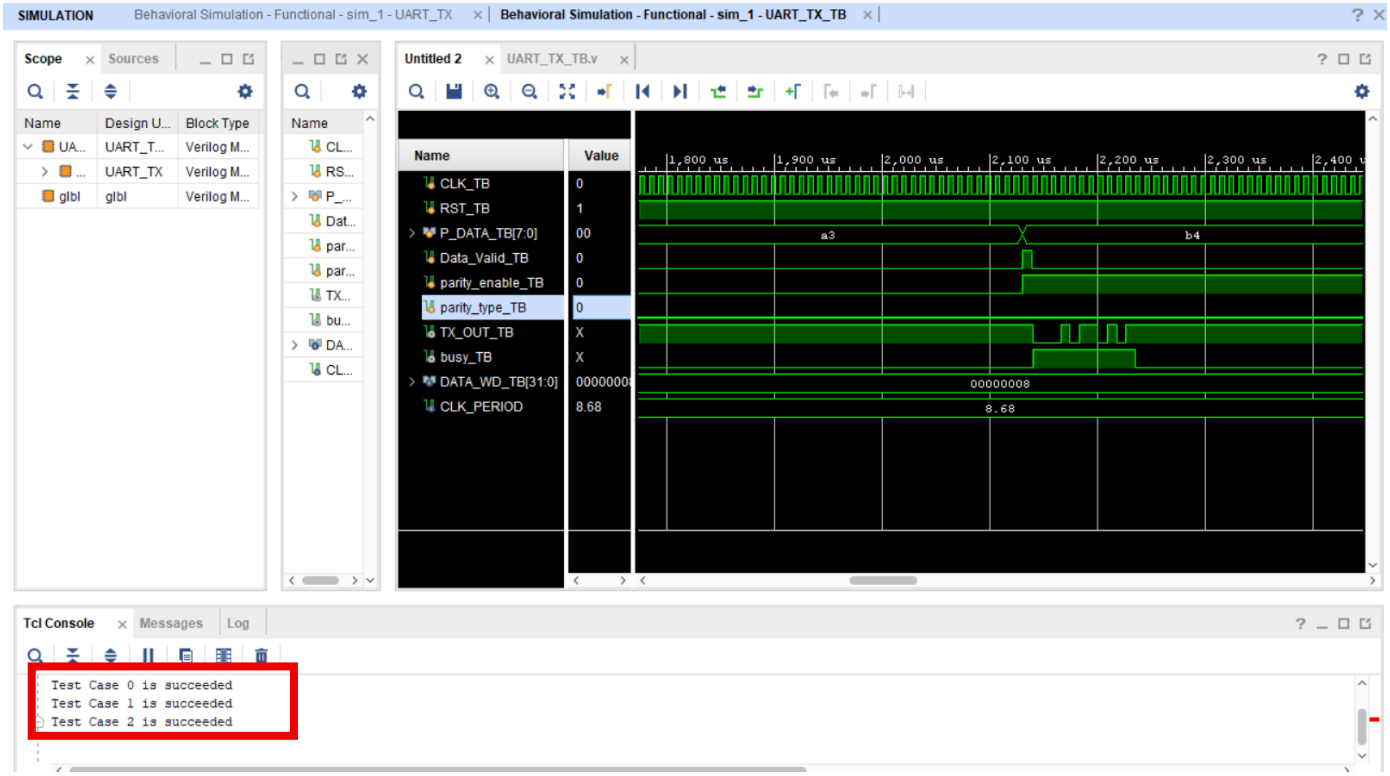
- Your test bench will be added to the simulation sources
- You can run simulation by:
  - Right click on simulation
  - choose Run Behavioral Simulation



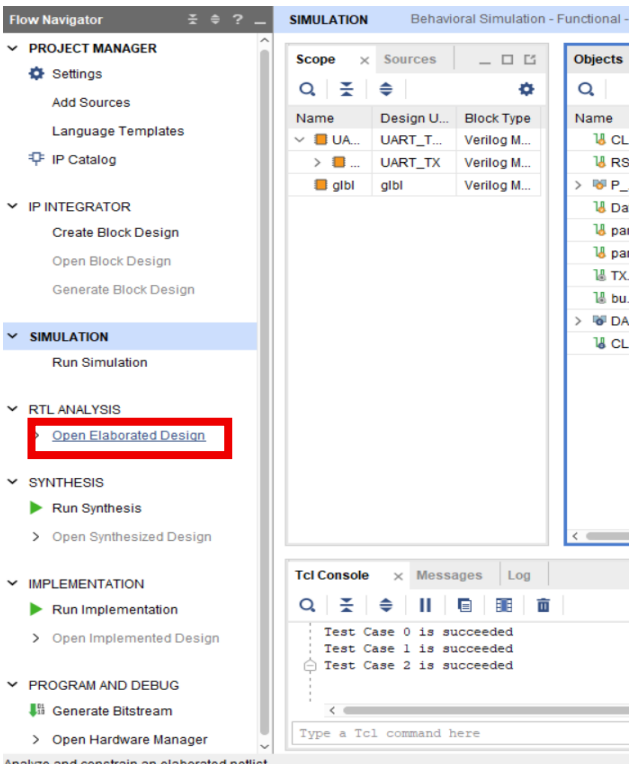
- Run simulation (click on **run all** in the tool bar)



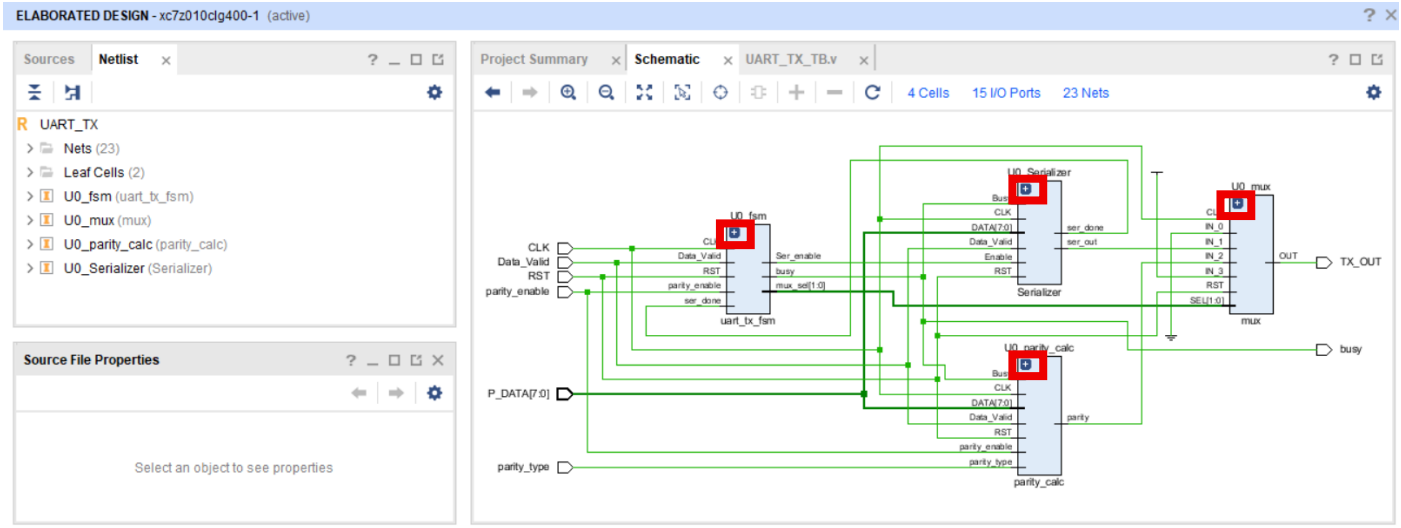
- You can debug the wave and the output terminal as shown below



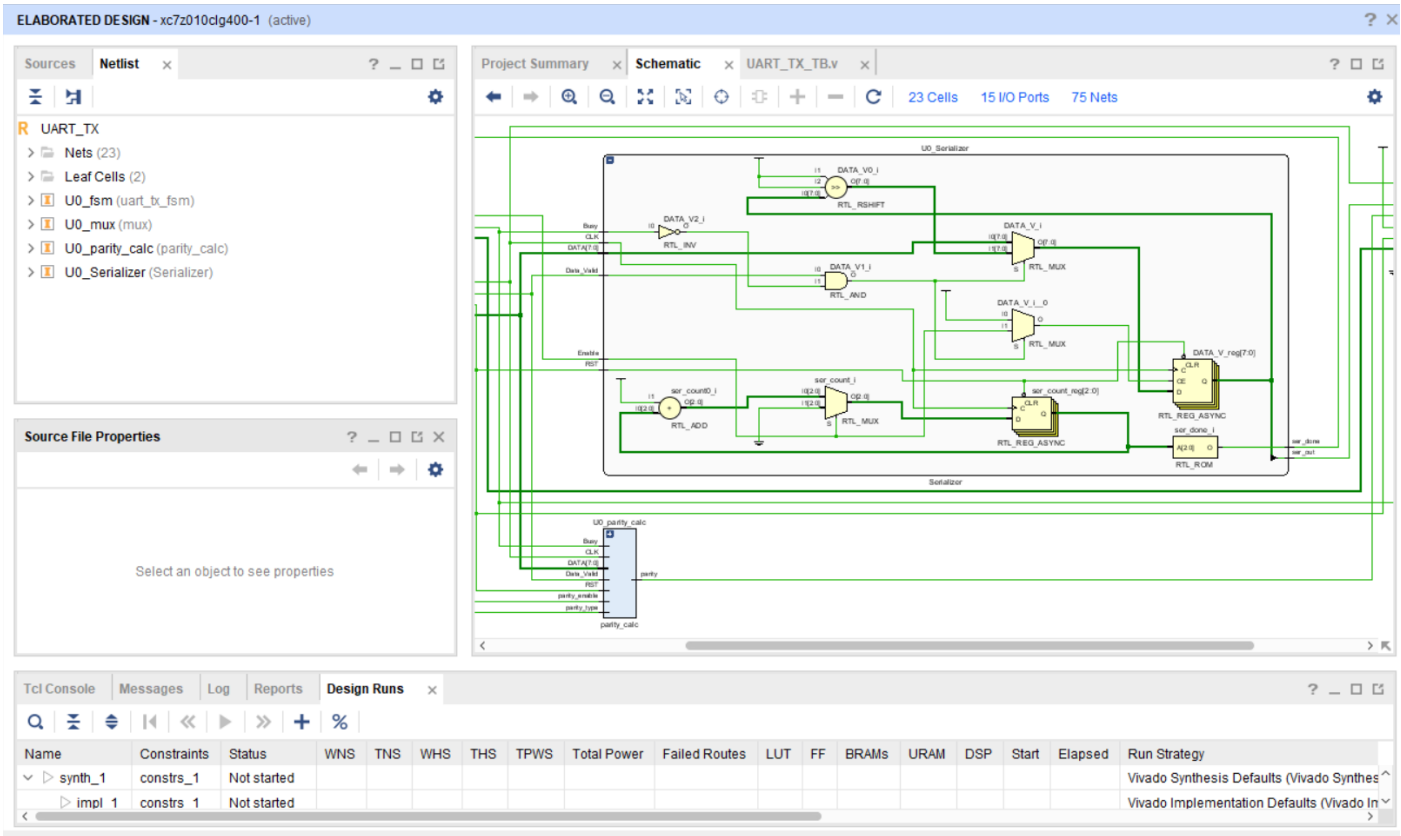
9. Run **Elaboration**



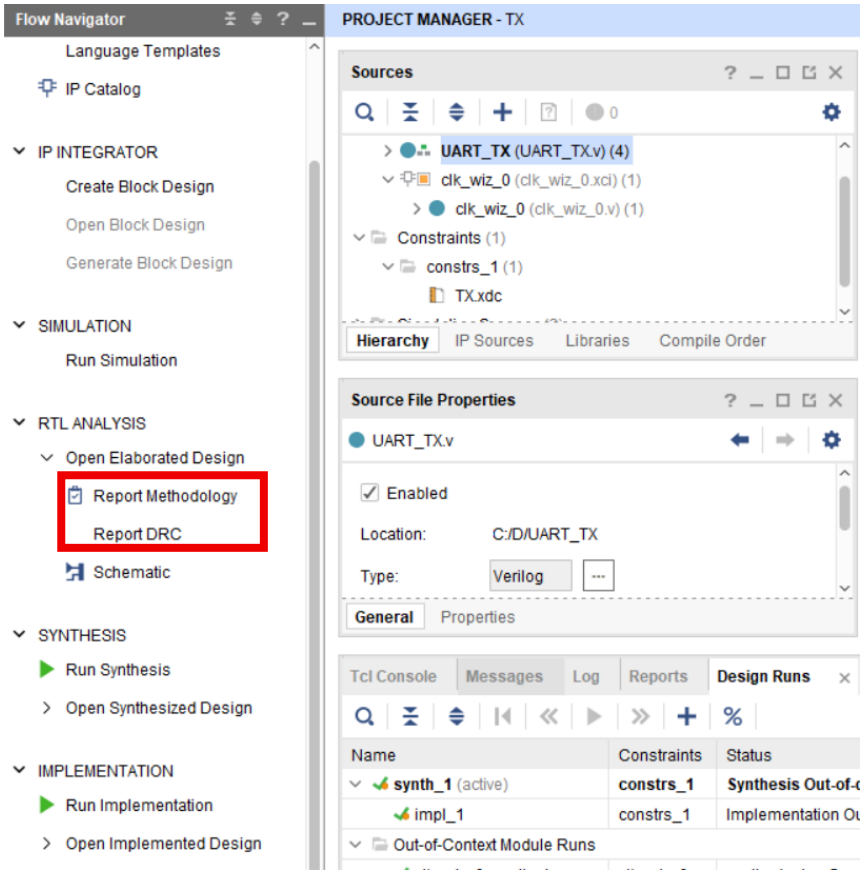
- After you run elaboration, you will see the following high-level schematic for your design (you can click on “+” sign on any block to see more details)

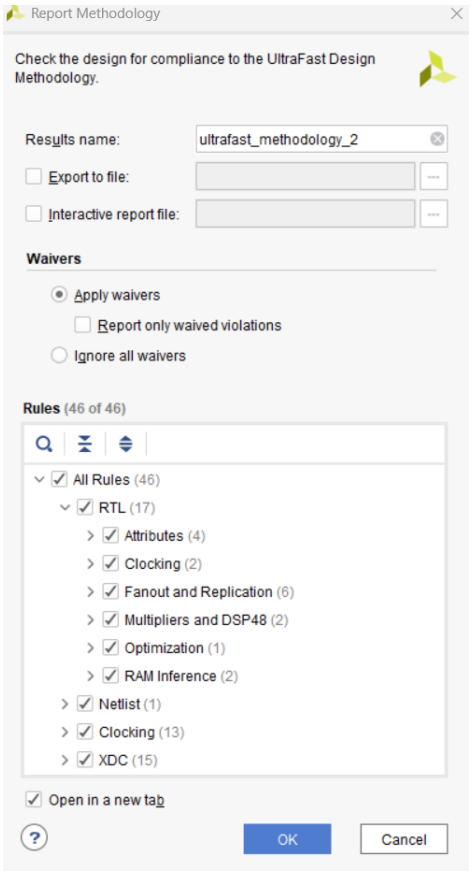


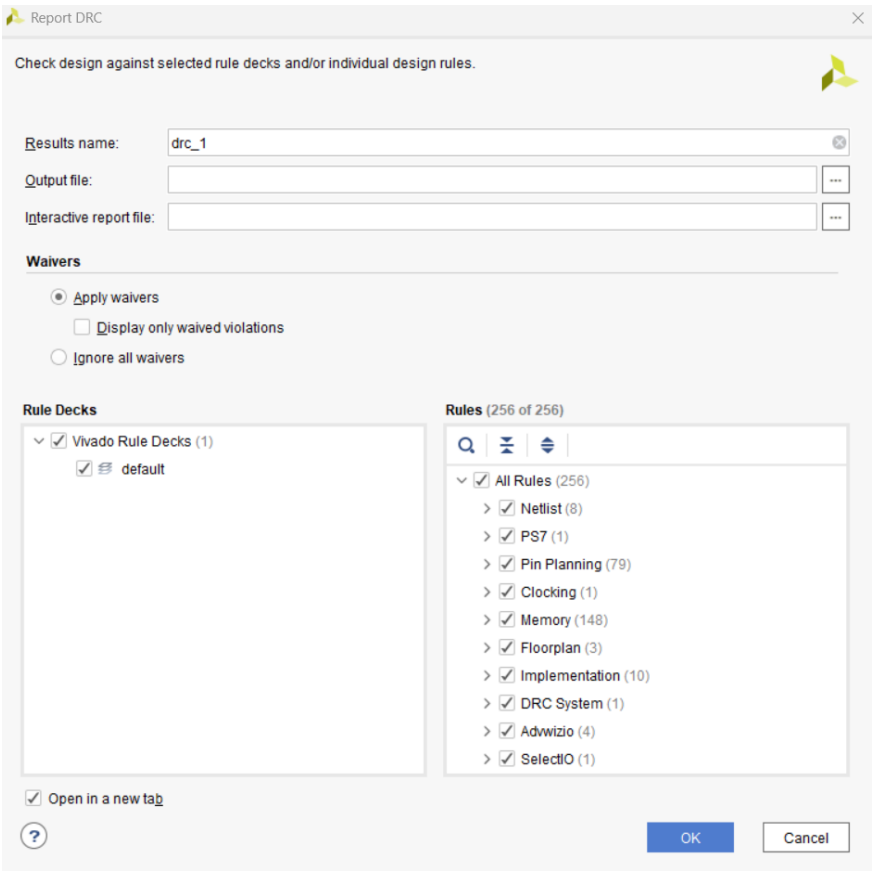
- If you click on “+” sign, this will be shown:



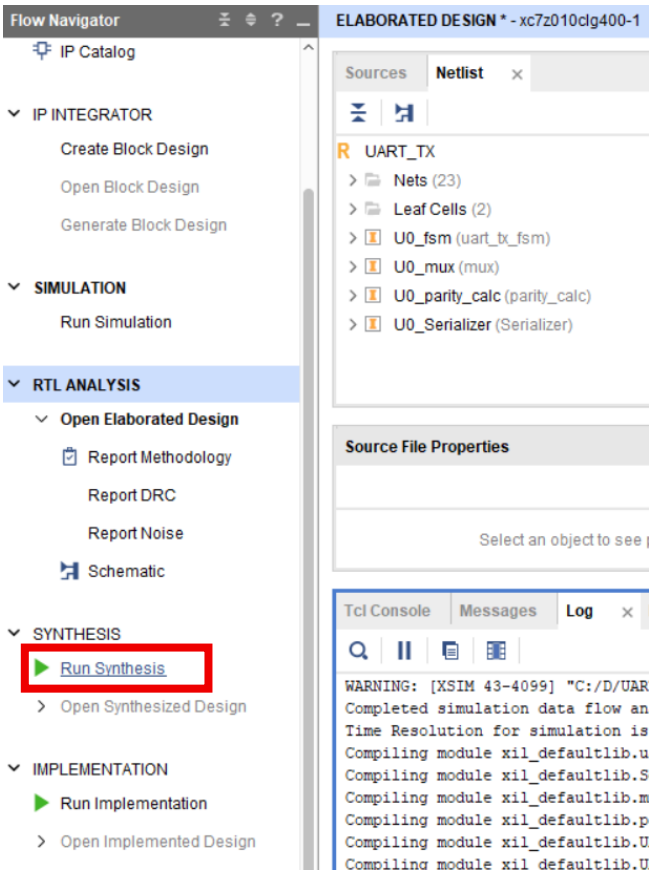
- You can run **Report Methodology** to identify potential design and implementation issues
- Also, you can run **Report DRC** to check for violations of Vivado's design rules, such as clocking, I/O, placement, and routing issues.



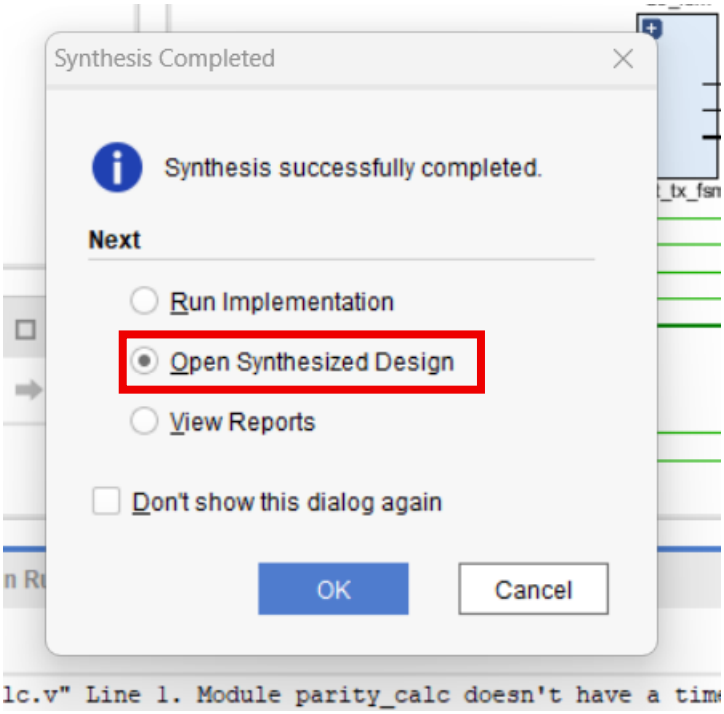




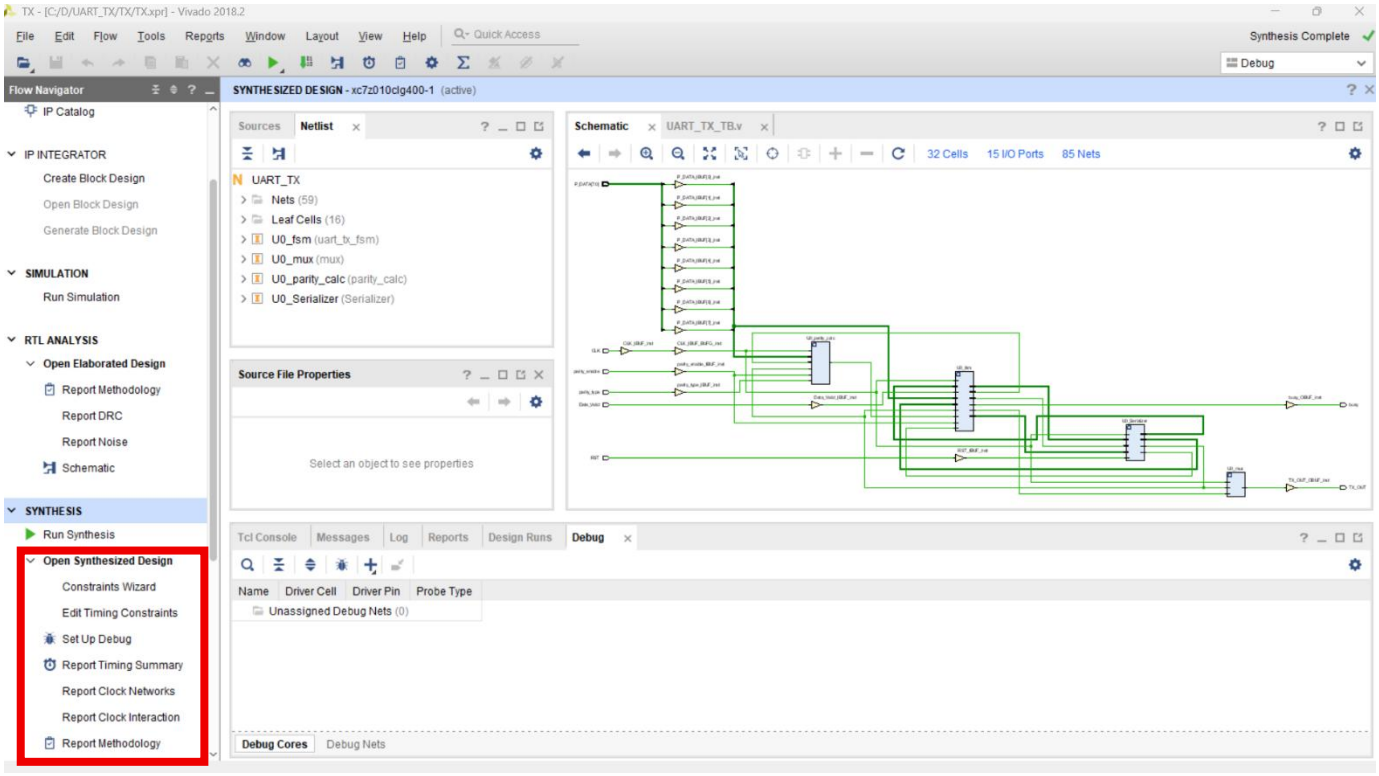
10. Run **Synthesis** to convert the RTL into a synthesized netlist (click on Run Synthesis)



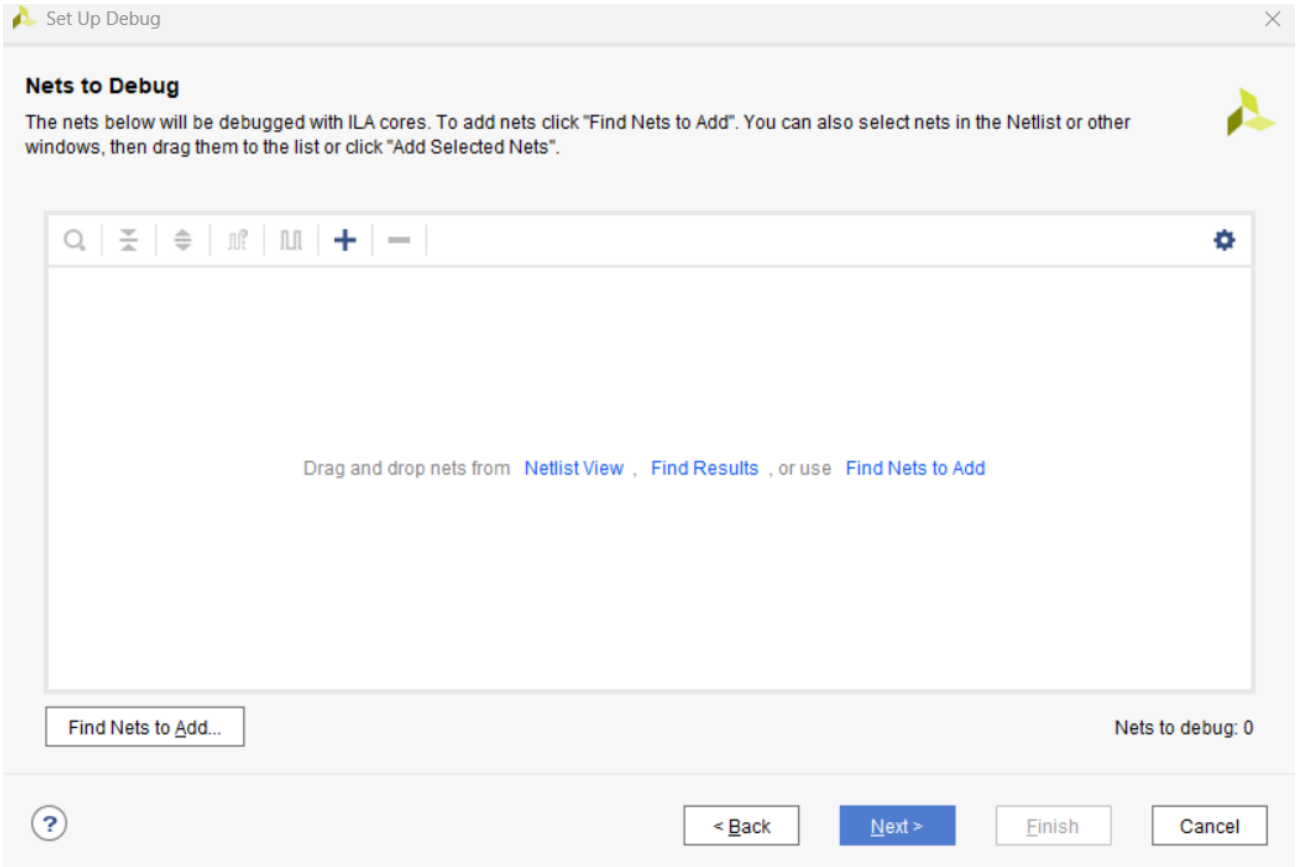
- Open the synthesized design



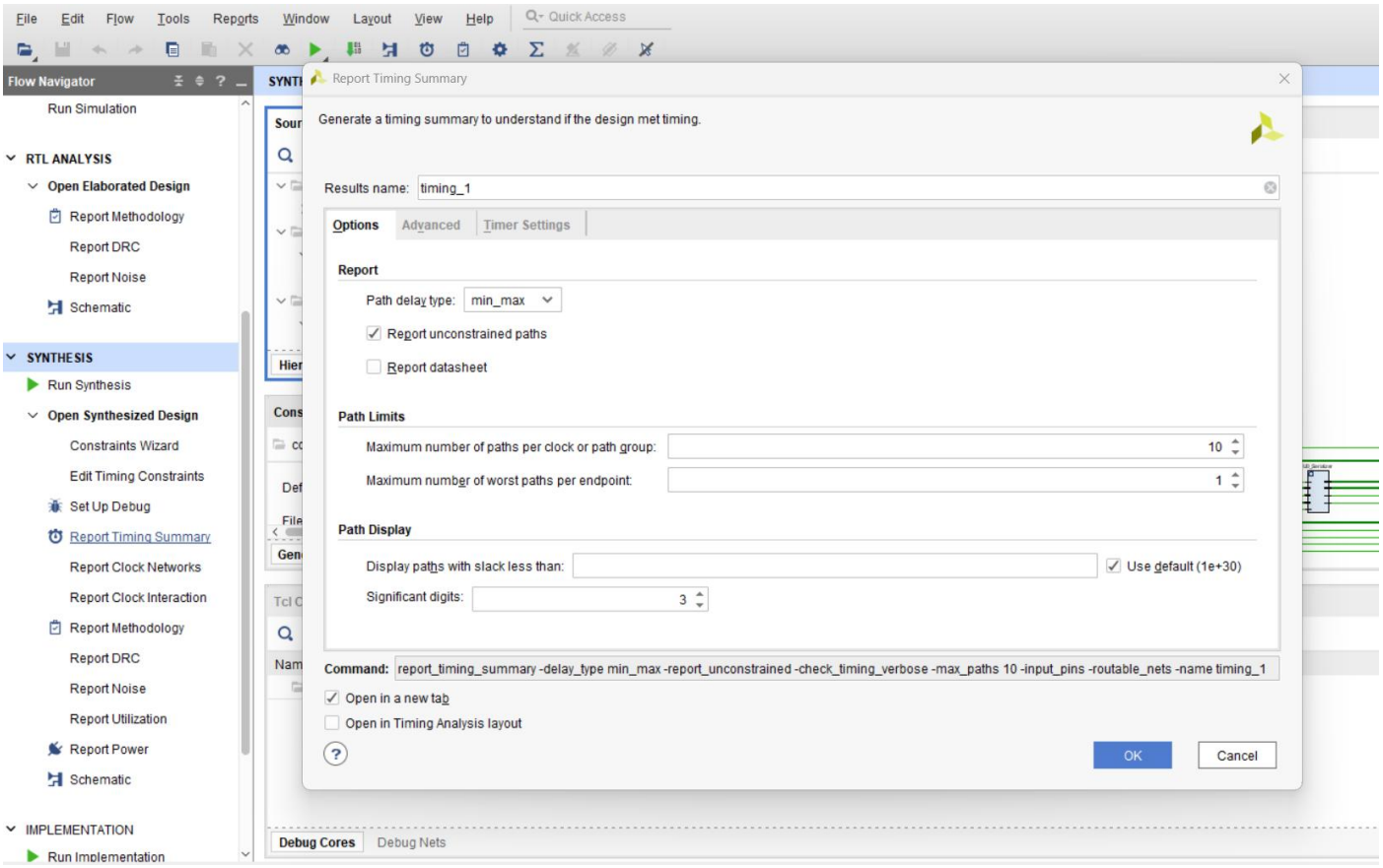
- The following more detailed schematic will be shown

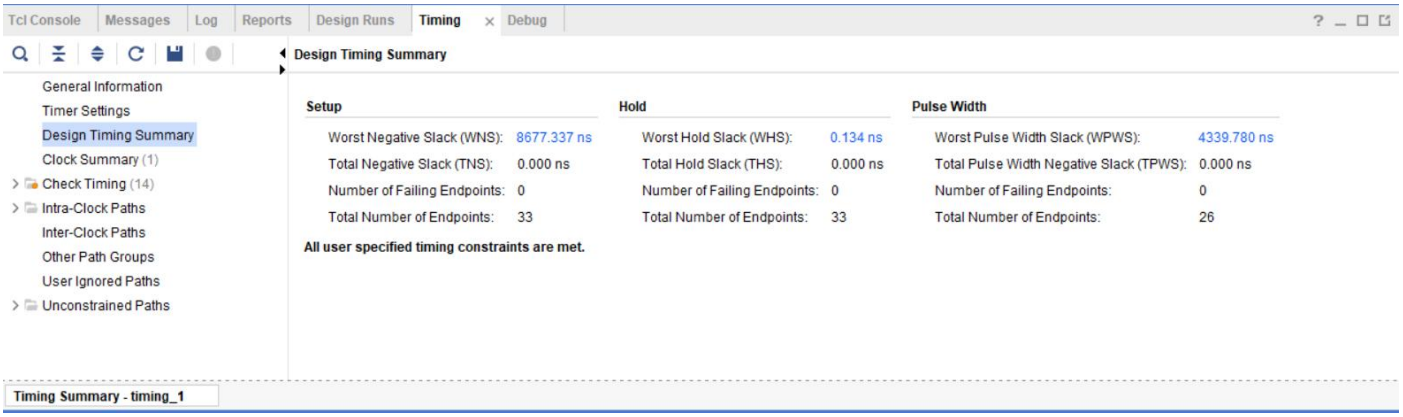


- There are many reports and options available in synthesis you can select
- **Set Up Debug** to monitor and analyze internal signals using ILA (Internal Logic Analyzer). **Please, check the last pages.**

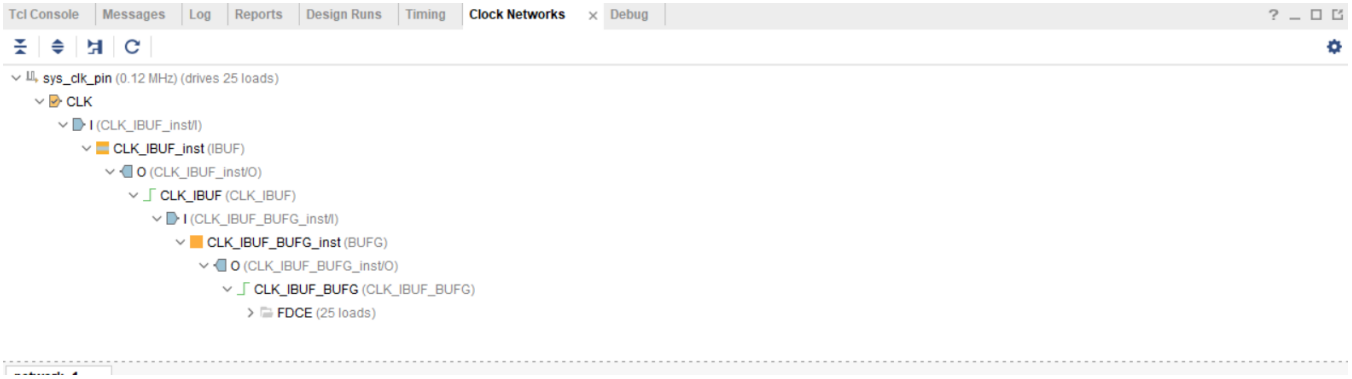


- **Report timing summary** to check setup and hold slacks

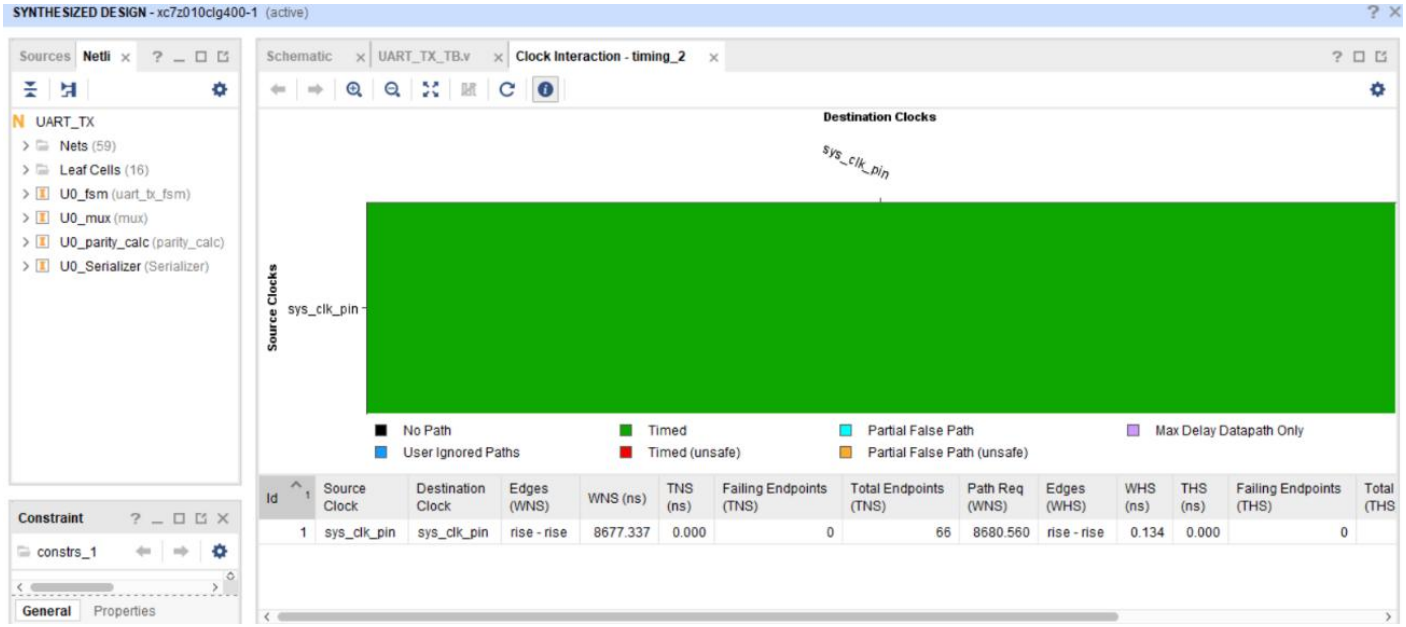




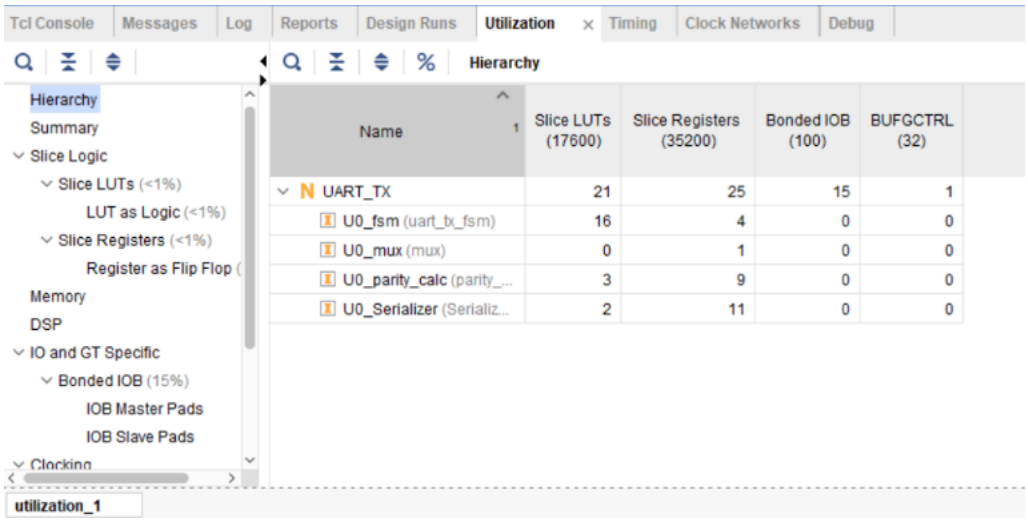
- **Report clock networks** to show the physical distribution tree of each clock



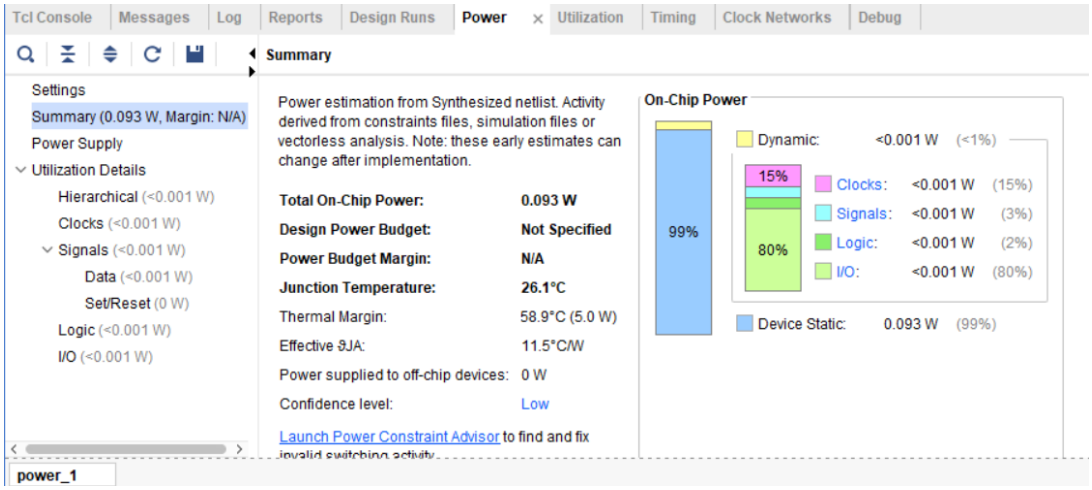
- **Report clock interaction** to show how each pair of clock domains relates (same source, related/unrelated, or asynchronous)



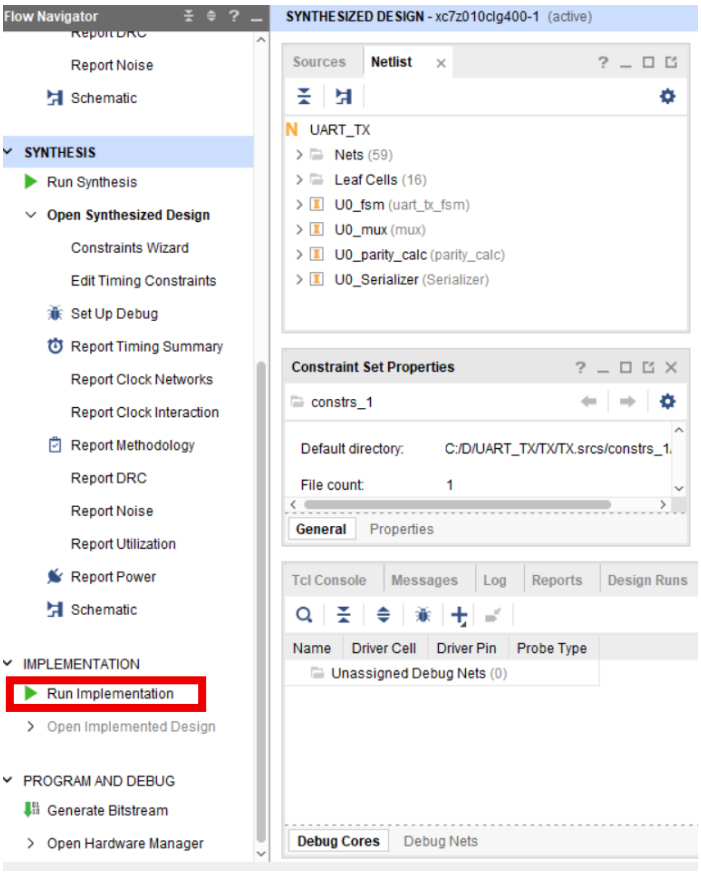
- **Report utilization** shows device resource usage (LUTs, FFs, BRAM, DSP, I/O, clocking) as Used vs. Available

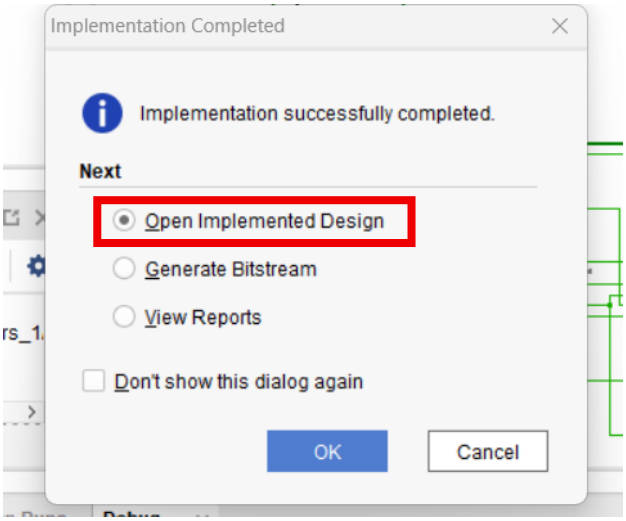


- **Report power** estimates total on-chip power (dynamic + static)

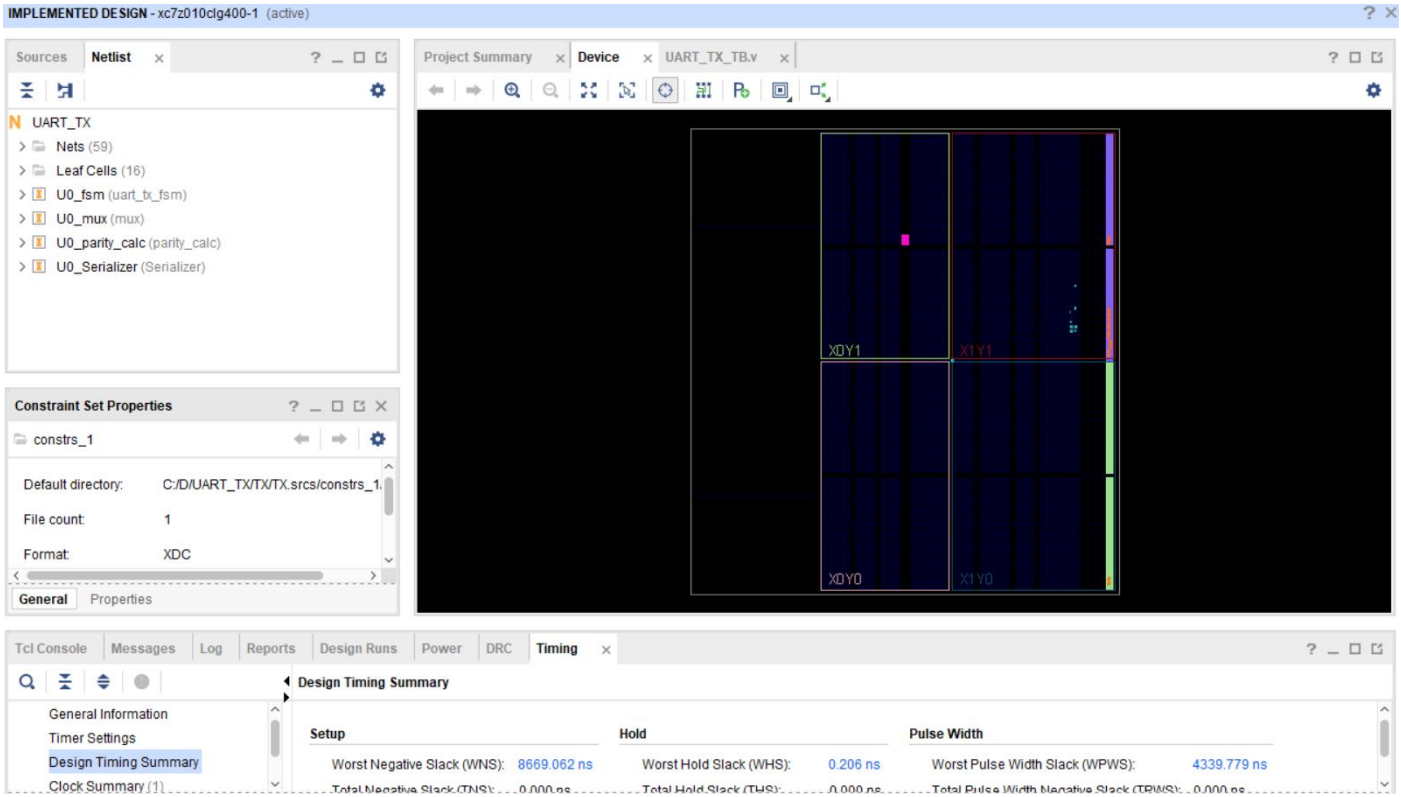


11. Run **Implementation** to perform placement and routing (click in run implementation)



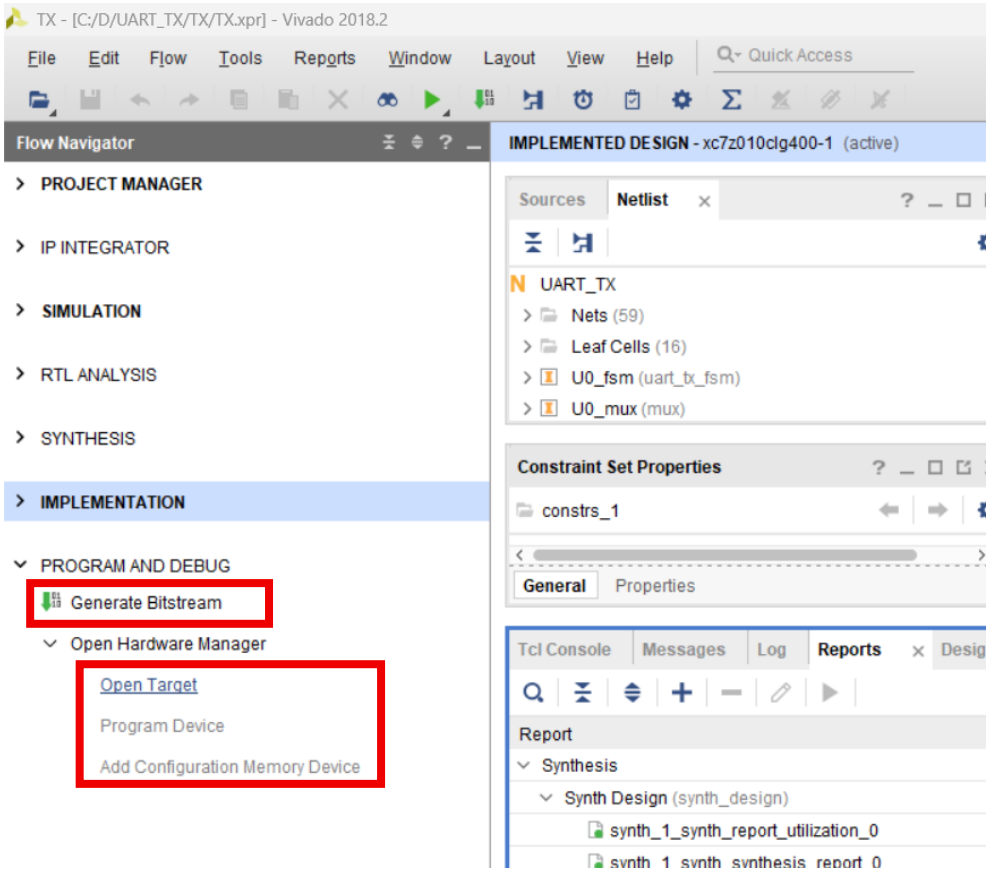


- The following window will be shown:

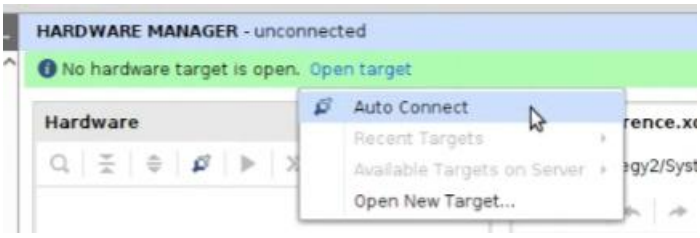


- You can check the same reports mentioned in synthesis but after implementation step.

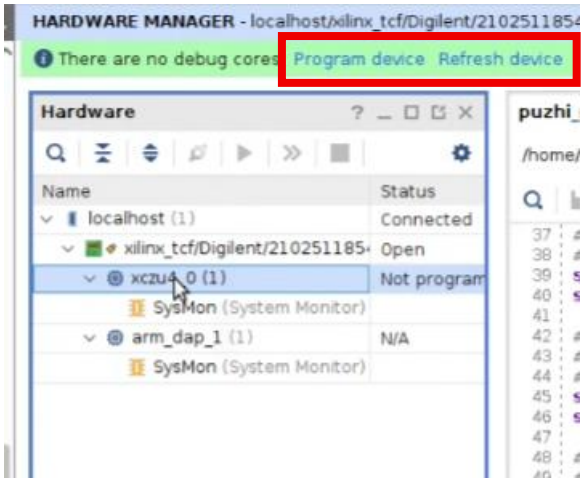
12. Generate the **bitstream** and use it to program the FPGA development board



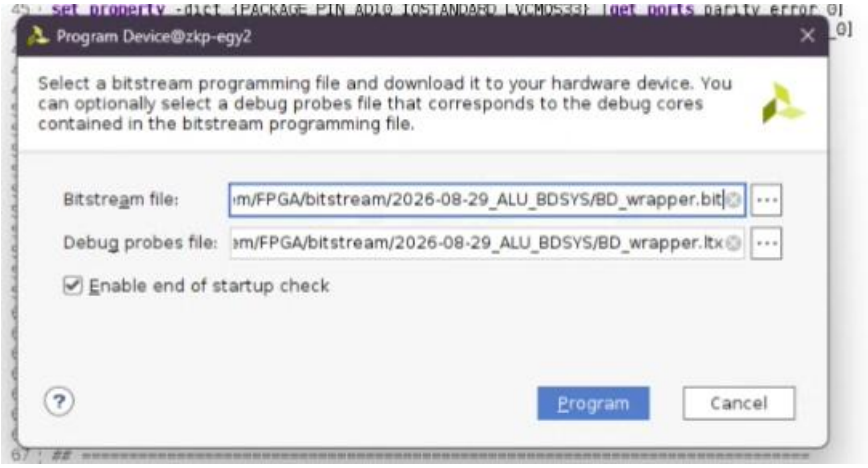
- After **Generate Bitstream** in Vivado, the flow is:
  - Open Hardware Manager
  - Open Target then Auto Connect



  - Vivado should detect your FPGA board through JTAG.
  - Program the FPGA
  - Click: **Program Device**.

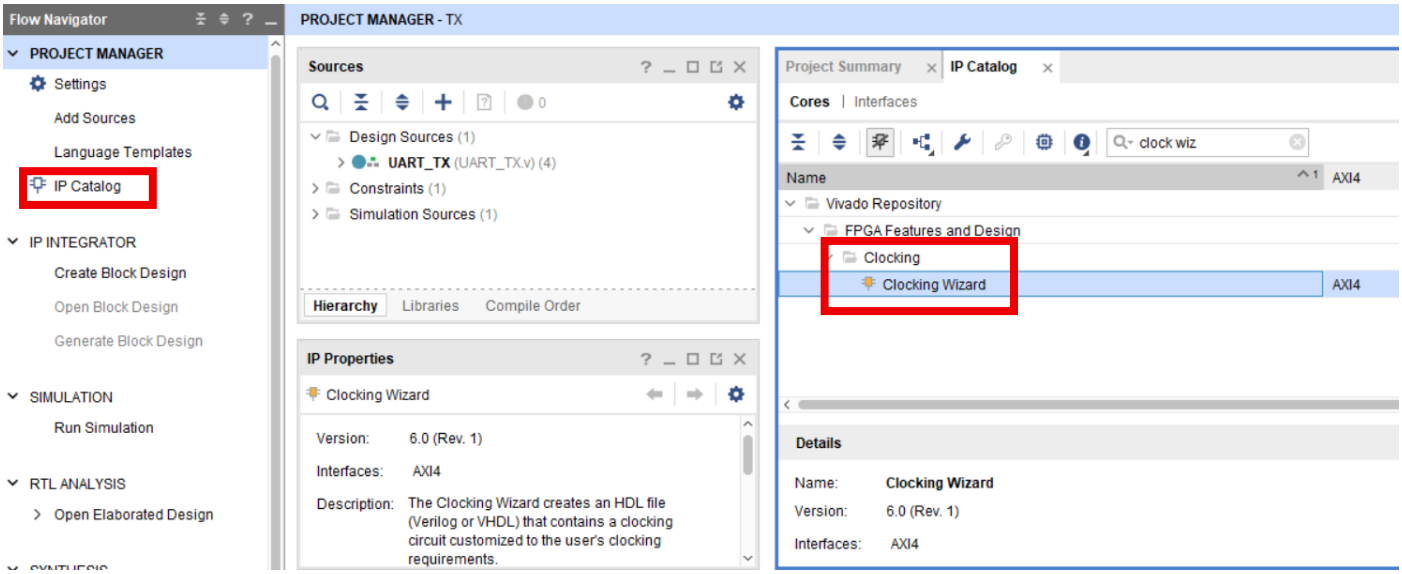


- Select your generated .bit file. Vivado normally finds it automatically.
- Then click **Program**. Your FPGA is now configured with your design

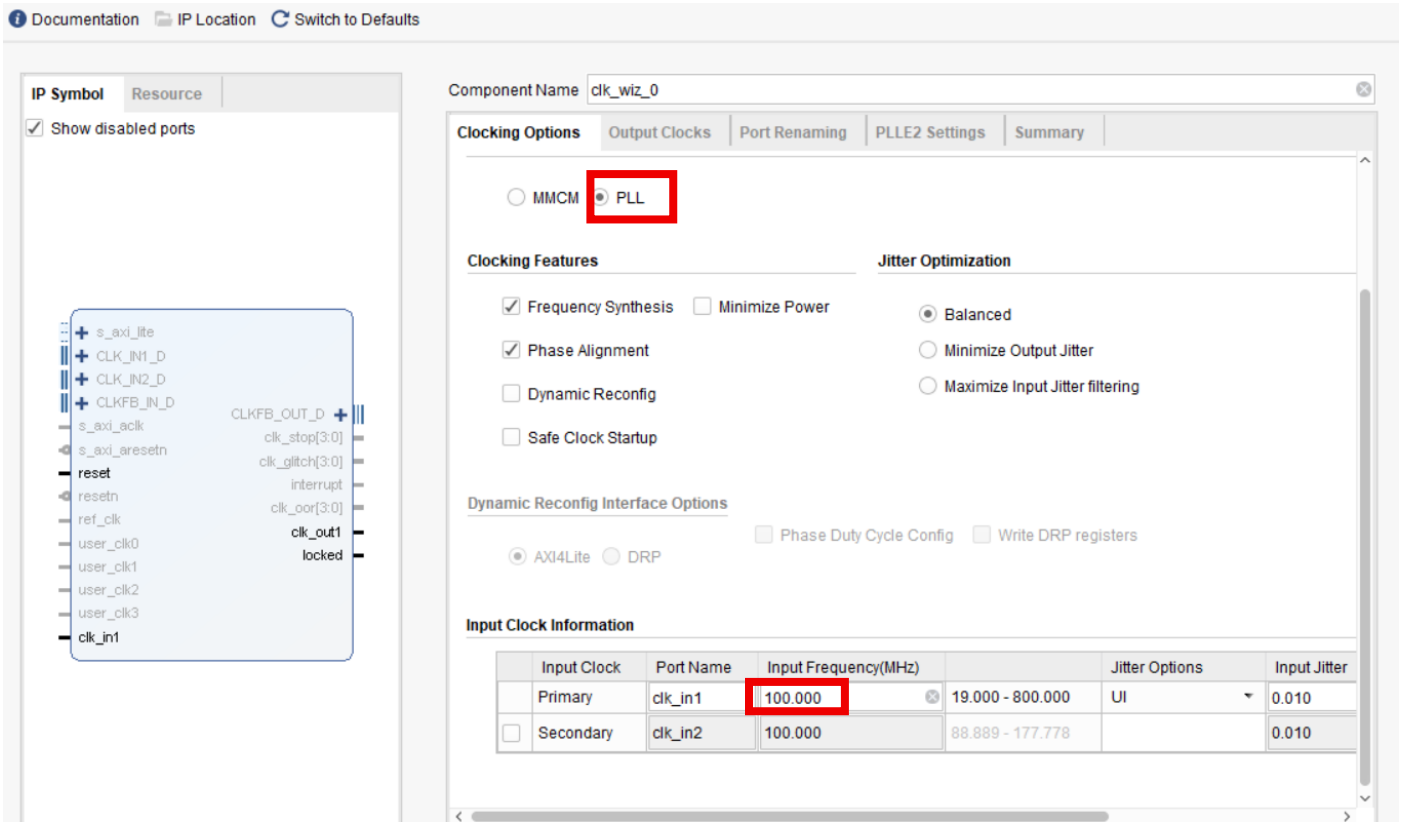


Steps to add a PLL IP to generate and control the clock frequency

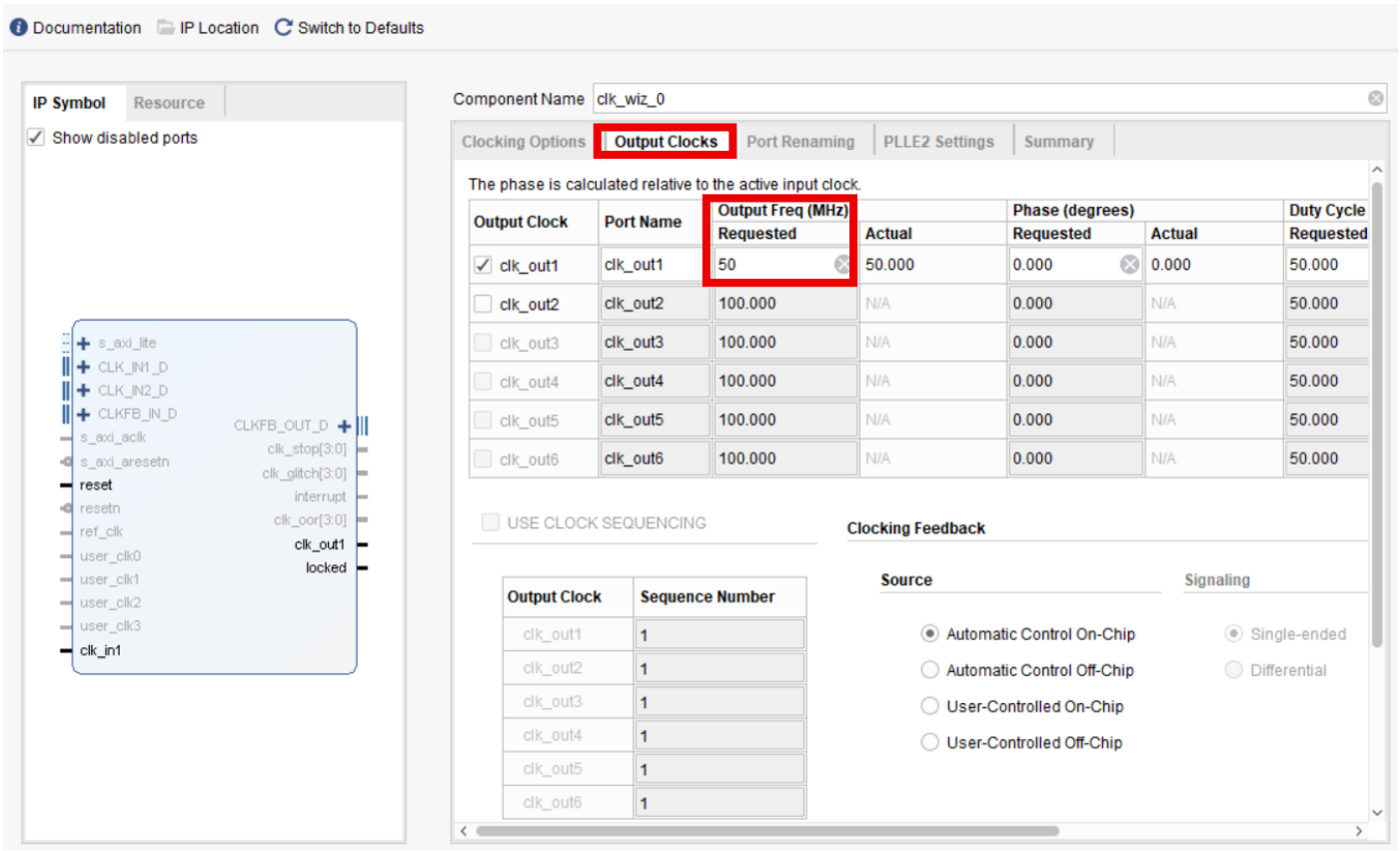
1. Open the clocking wizard IP to configure the PLL IP



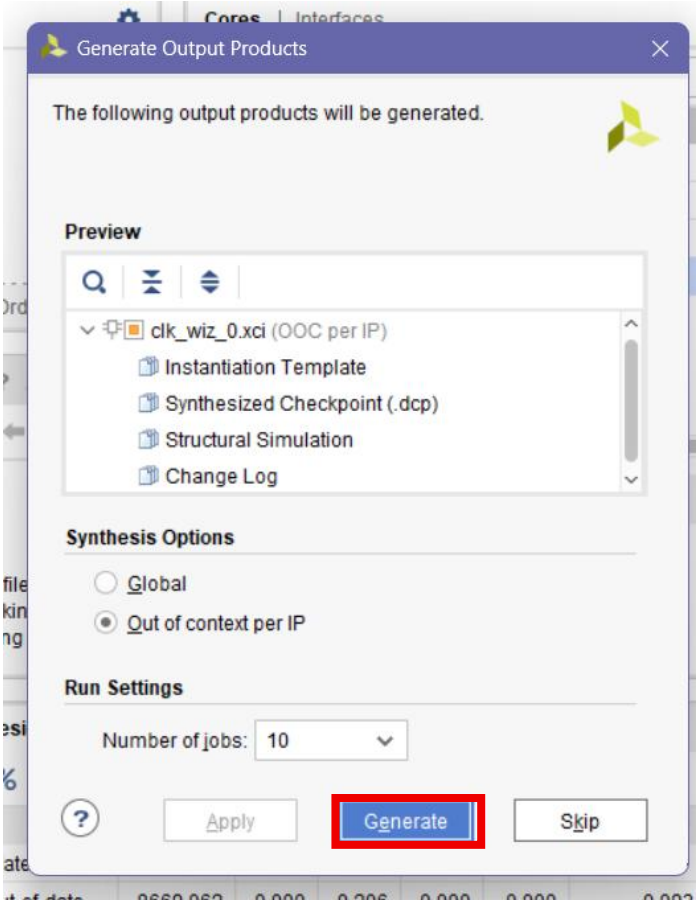
2. Choose PLL. The input clock to the PLL from the crystal oscillator is 100 MHz



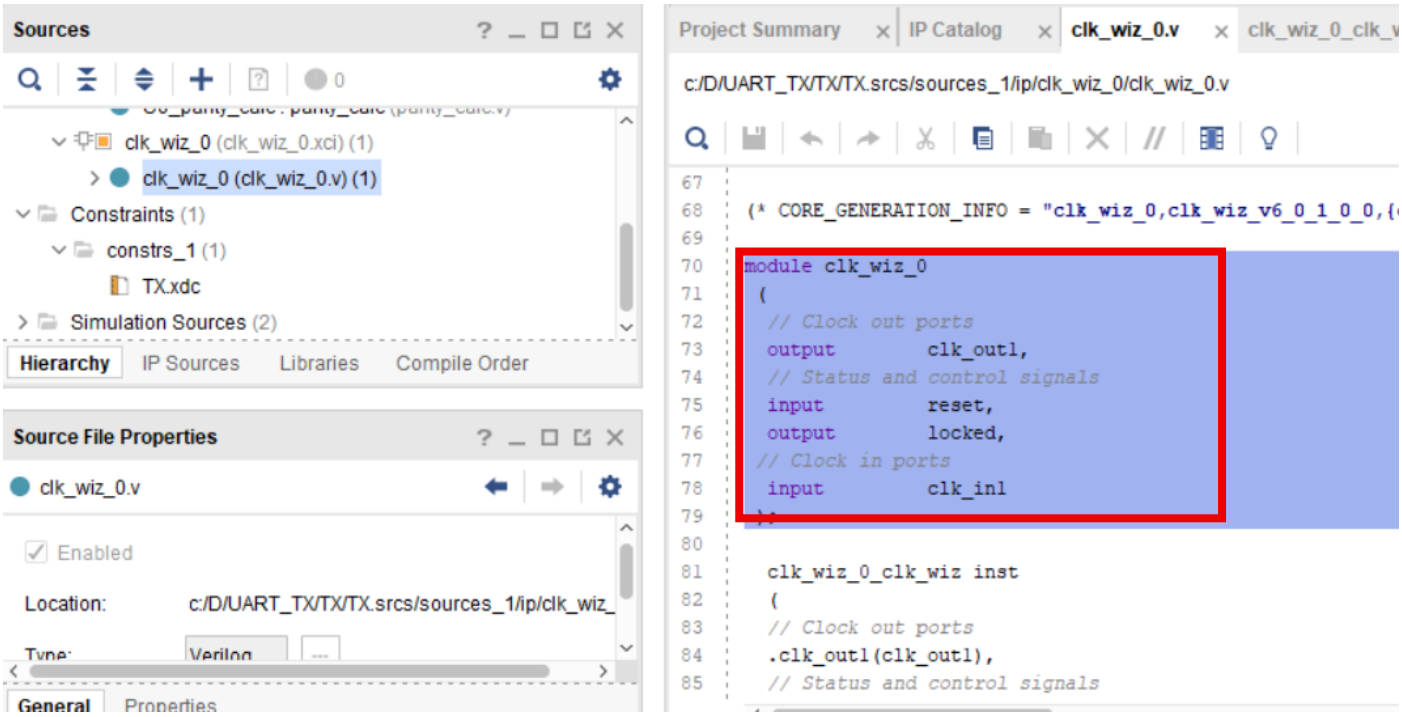
3. Choose the desired output frequency and number of output clocks



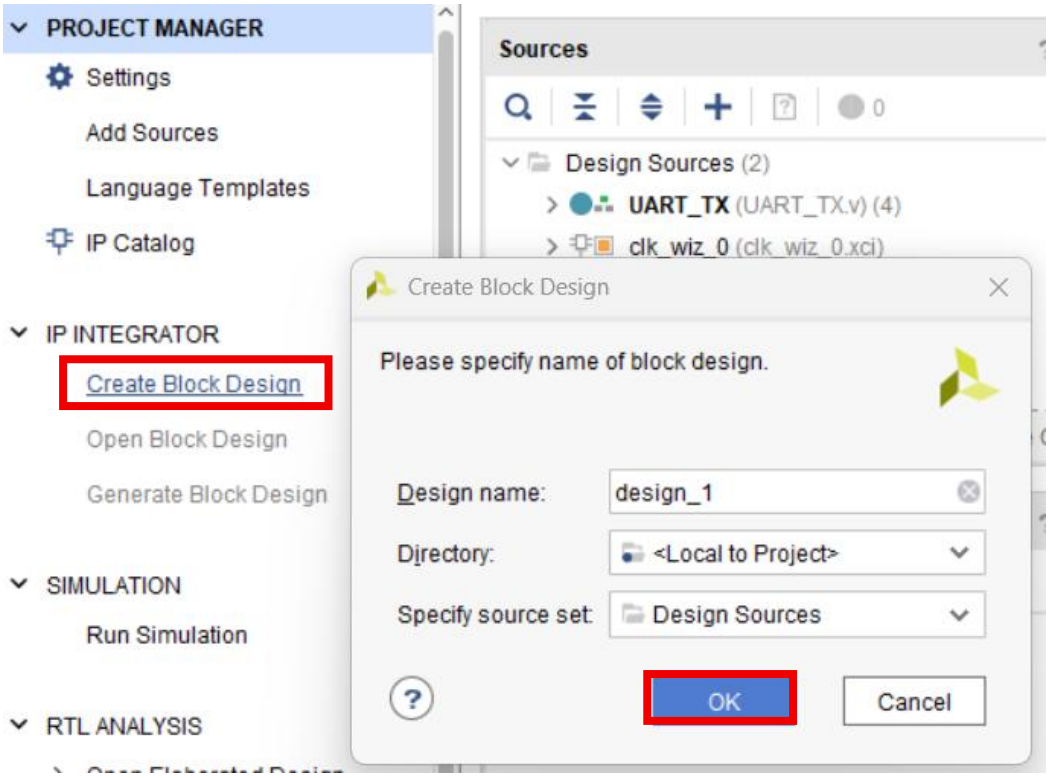
4. Generate the IP



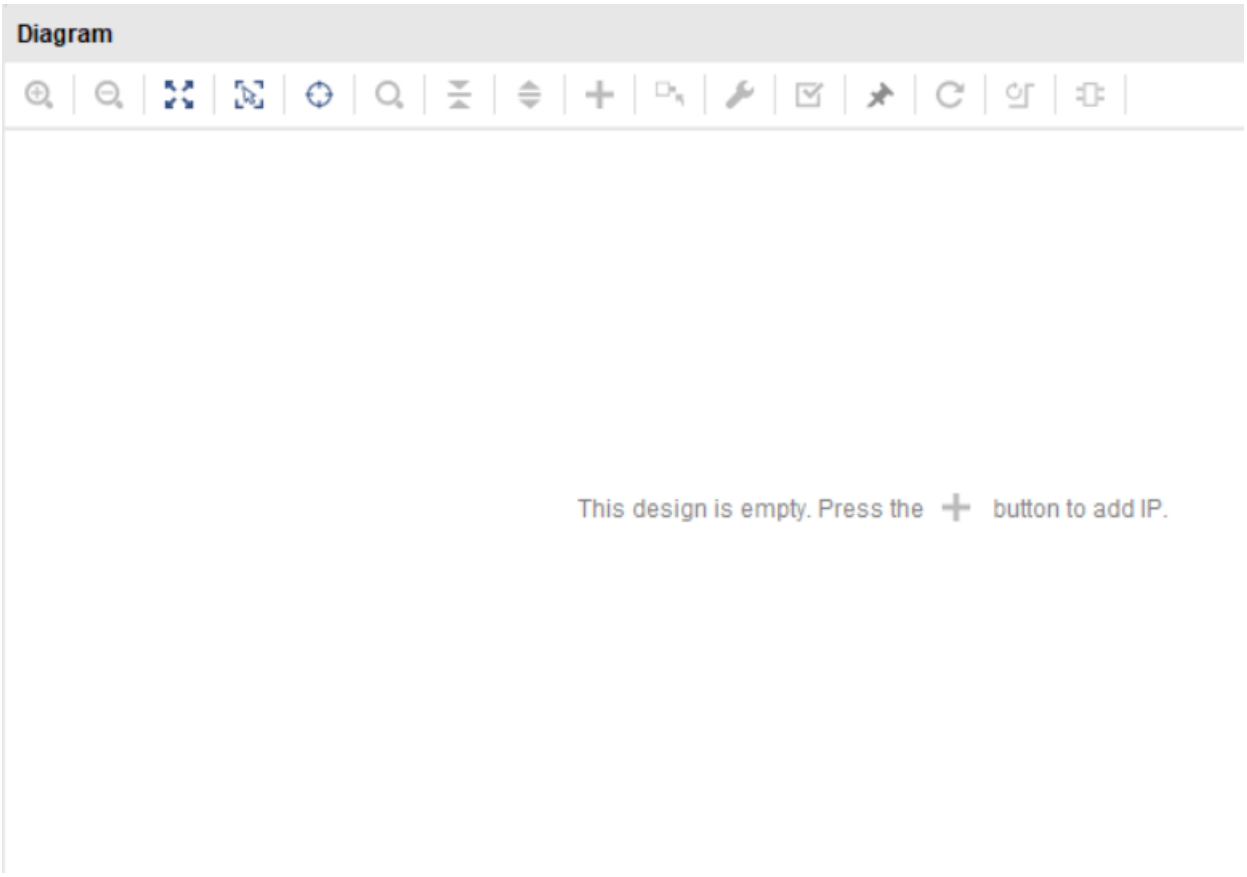
- IP netlist to be used for instantiation

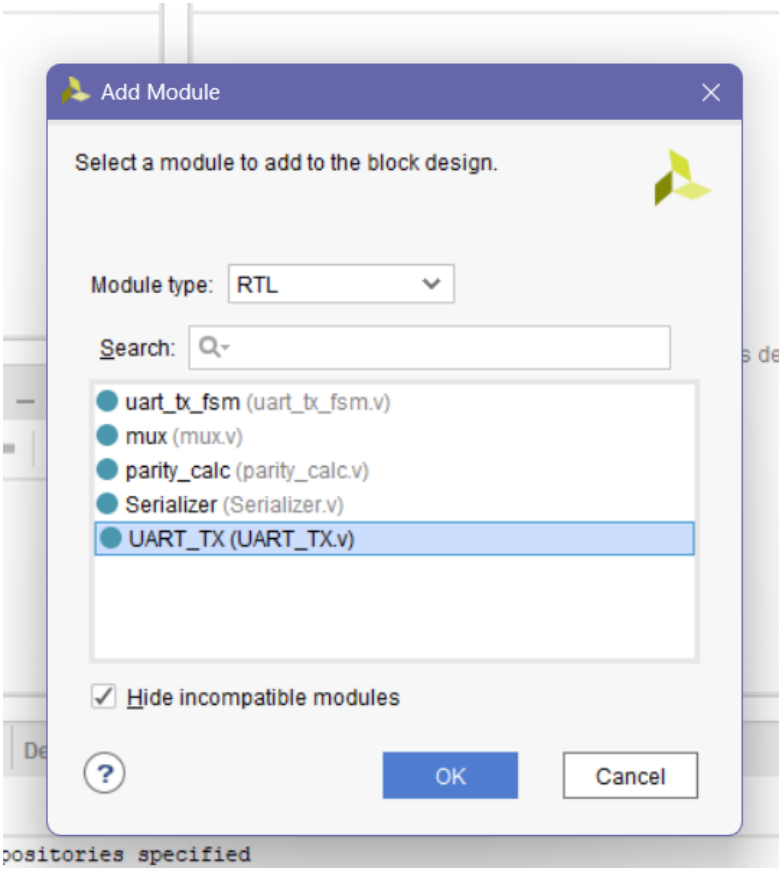


- **Another way** to do this is to use the IP integrator directly, create your block diagram and make the tool build the top wrapper:

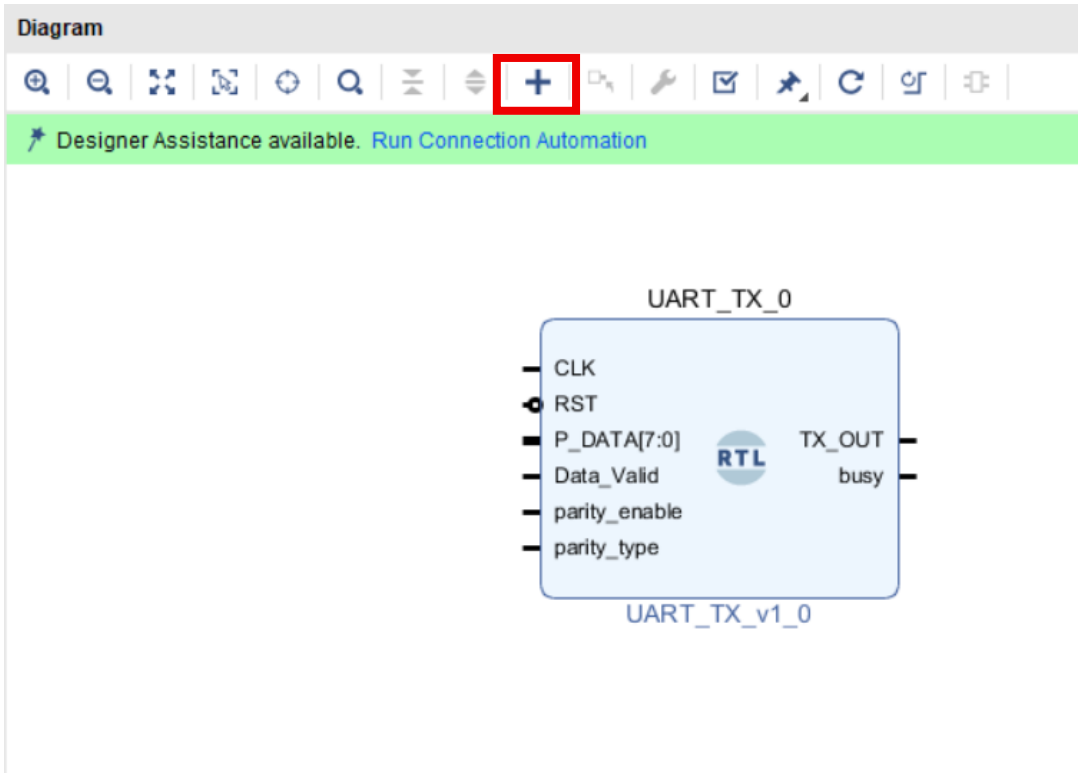


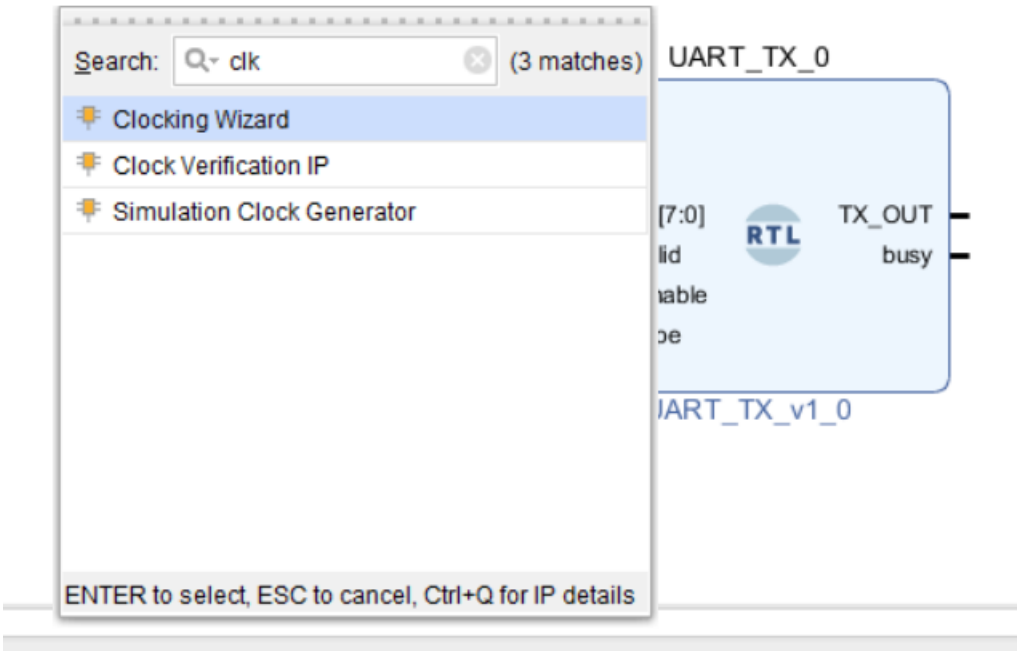
- Right click on the following empty space then Add module and select your top module



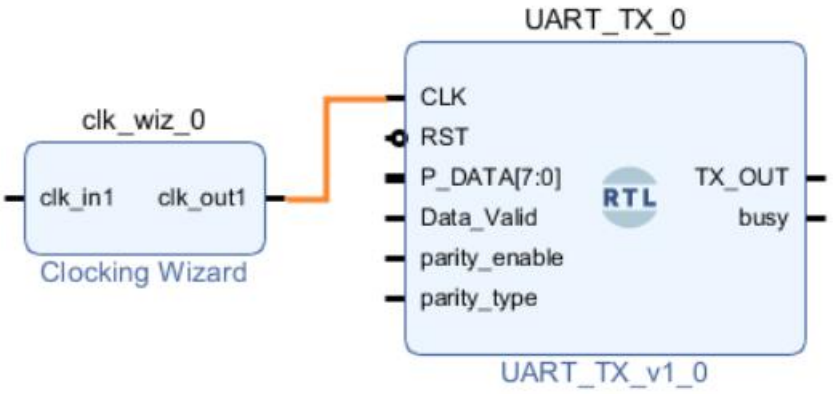


- The following image will be shown to you (click on “+” to add an IP):

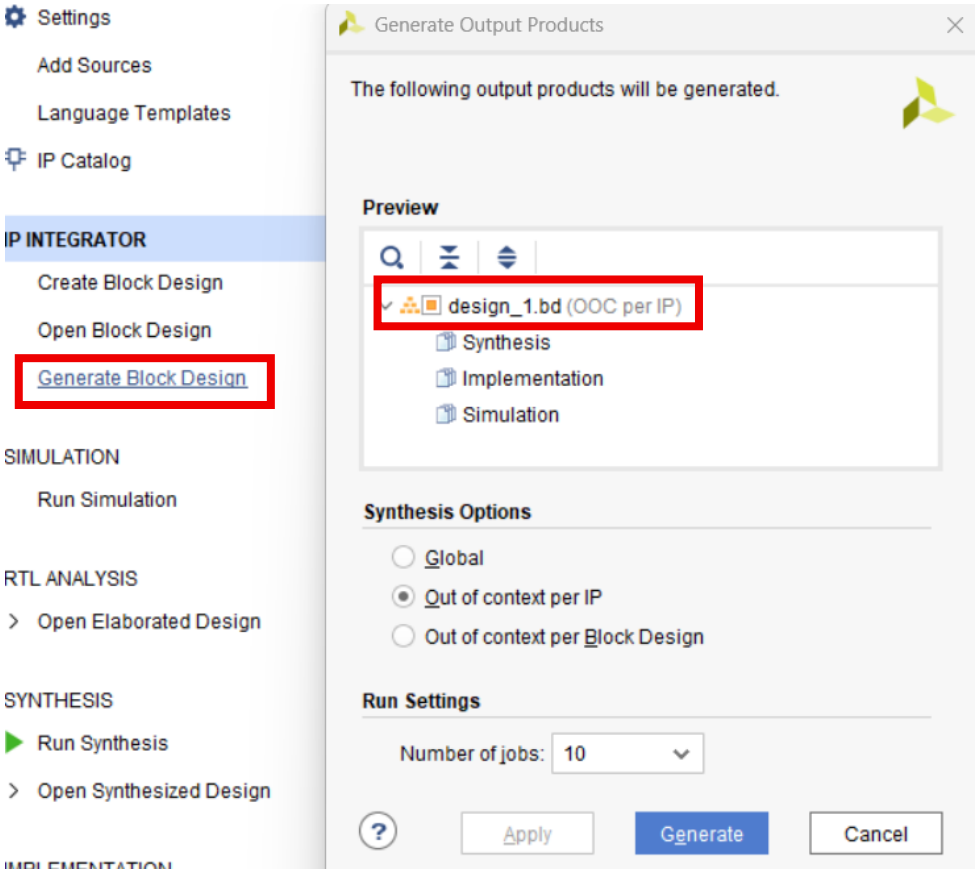




- Connect the 2 blocks then **right click** on each input and output and select **Make External**

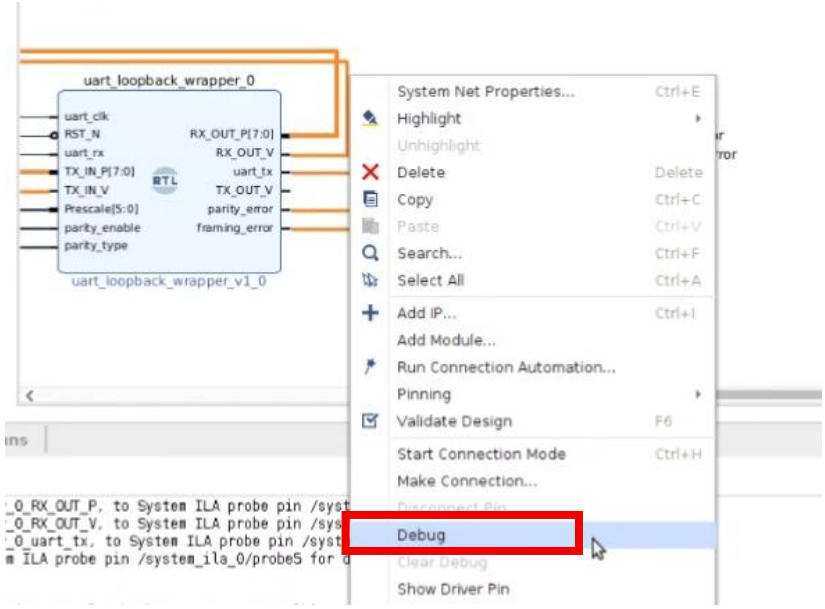


- After you finish this, click on **generate block design** to generate the HDL Wrapper. Also, you can do this by **right clicking on the .bd** file and selecting **create HDL wrapper**

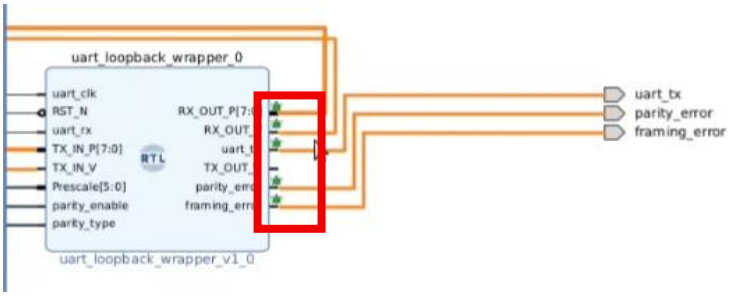


Steps for the ILA (Internal Logic Analyzer):

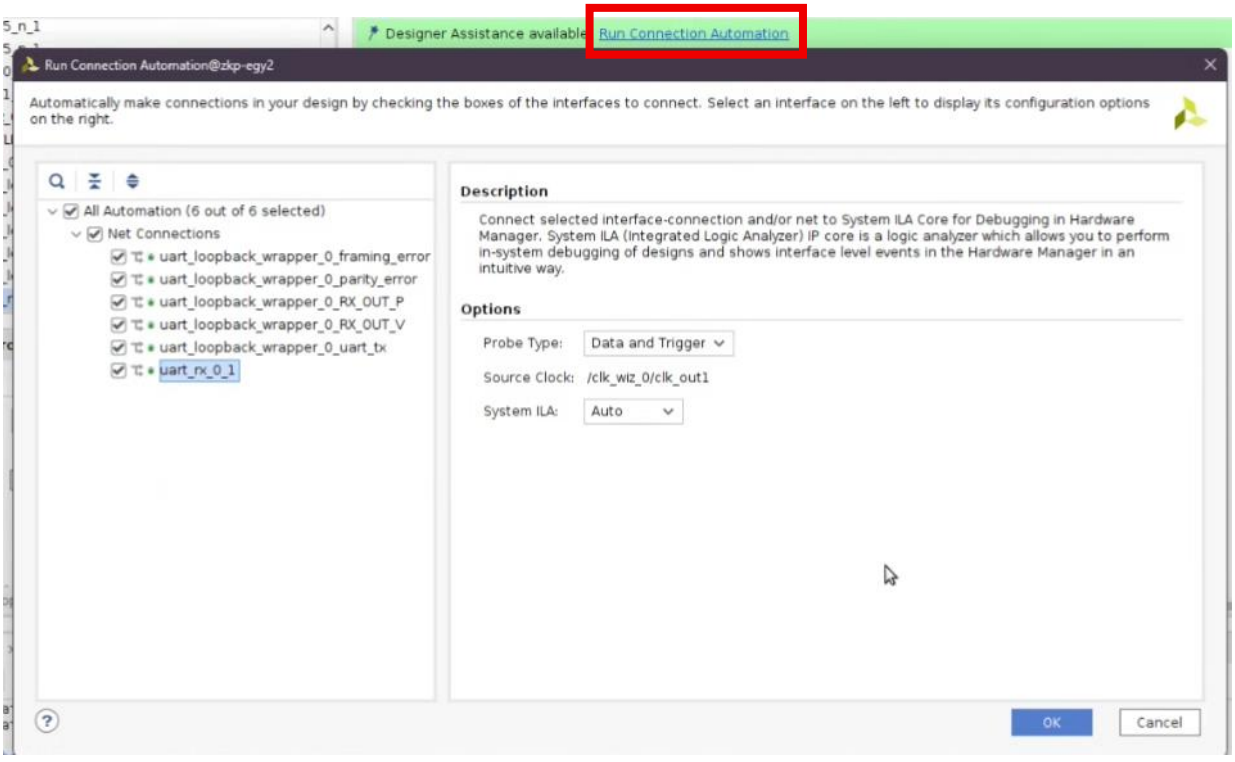
- **Right click** on any port you want to trace after deploying on the FPGA then **select Debug**



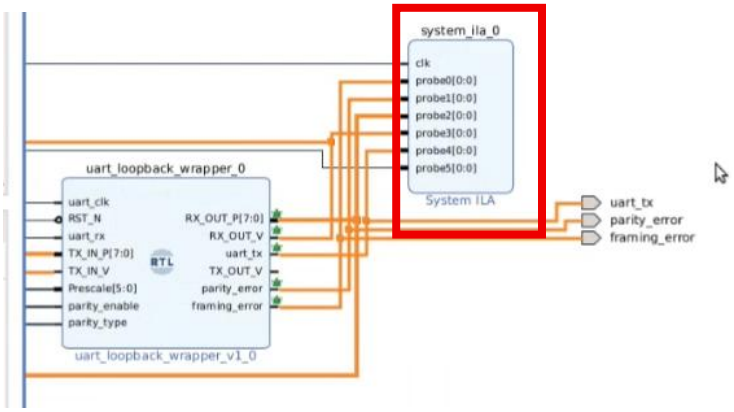
- The following stars should be added to your selected ports



- Run Connection Auntomation then Ok



- The ILA will be shown so we can debug these ports while running FPGA
- **Note**: You should connect the ILA to a clock faster than the ports you are monitoring.



- These ports will be shown later in the set up debug option
-

